

機器學習數學 Part 2

鄭宇倫

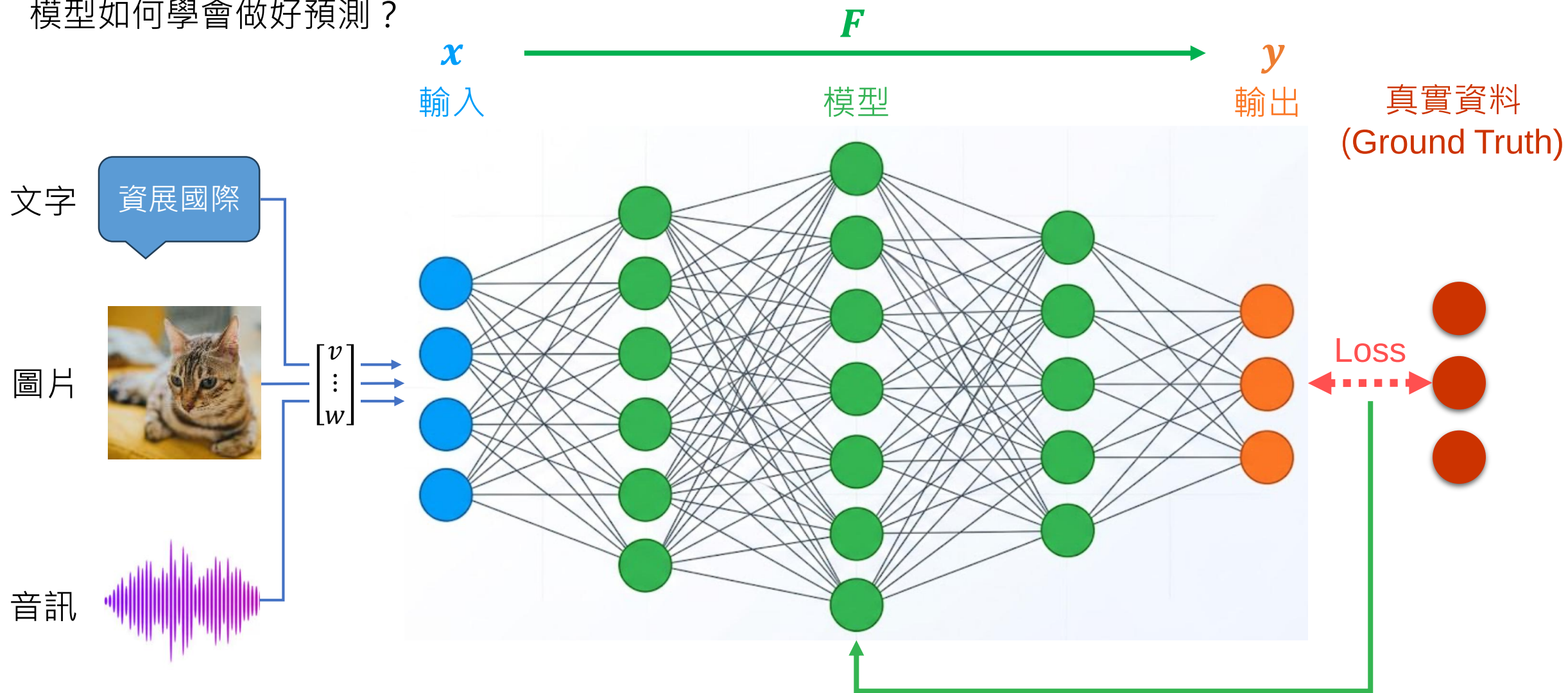
Chapter 2.

微分

1. 深度學習優化策略
2. 微分
3. 偏微分與梯度
4. 連鎖律
5. 反向傳播
6. 記憶體管理

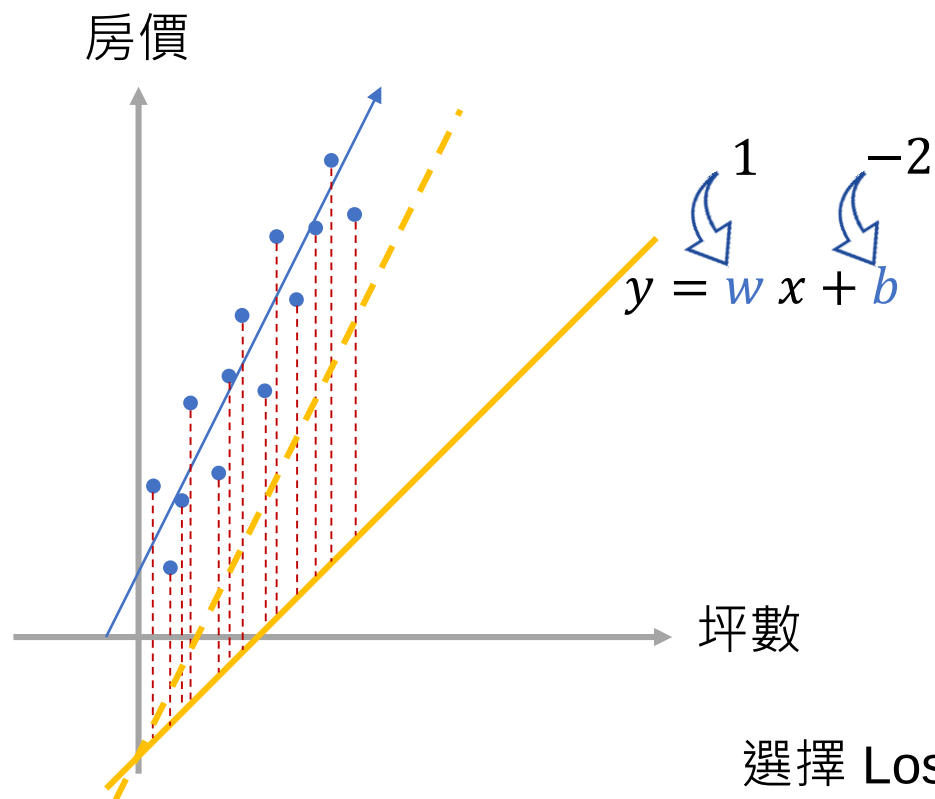
深度學習模型與訓練機制

- 模型如何學會做好預測？



線性迴歸與損失函數

Loss 如何量化？



- 數據呈線性分布

⇒ 選擇使用線性函數去預測

$$y = wx + b$$



模型參數，透過訓練求得

1. 先隨機選擇 w 和 b (初始化)
2. 計算「預測值與真實值的誤差」

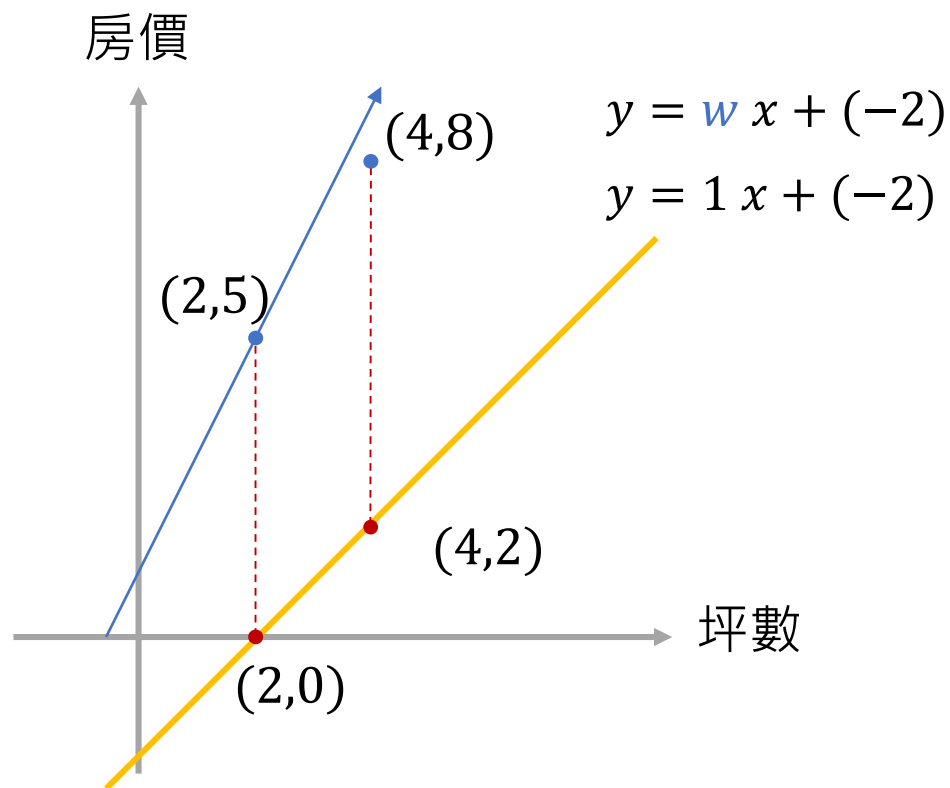
選擇 Loss 設計：MSE = mean squared error = $\sum_n (y_{\text{true}} - y_{\text{pred}})^2 / n$

3. 調整 w 和 b ，降低 Loss

HOW??

單變數損失函數建構

簡化問題：把誤差變成單變數



- 假設只有兩筆資料 $(2, 5)$ 、 $(4, 8)$

隨機初始化函數為 $y = 1x + (-2)$

則模型預測值為 $(2, 0)$ 、 $(4, 2)$

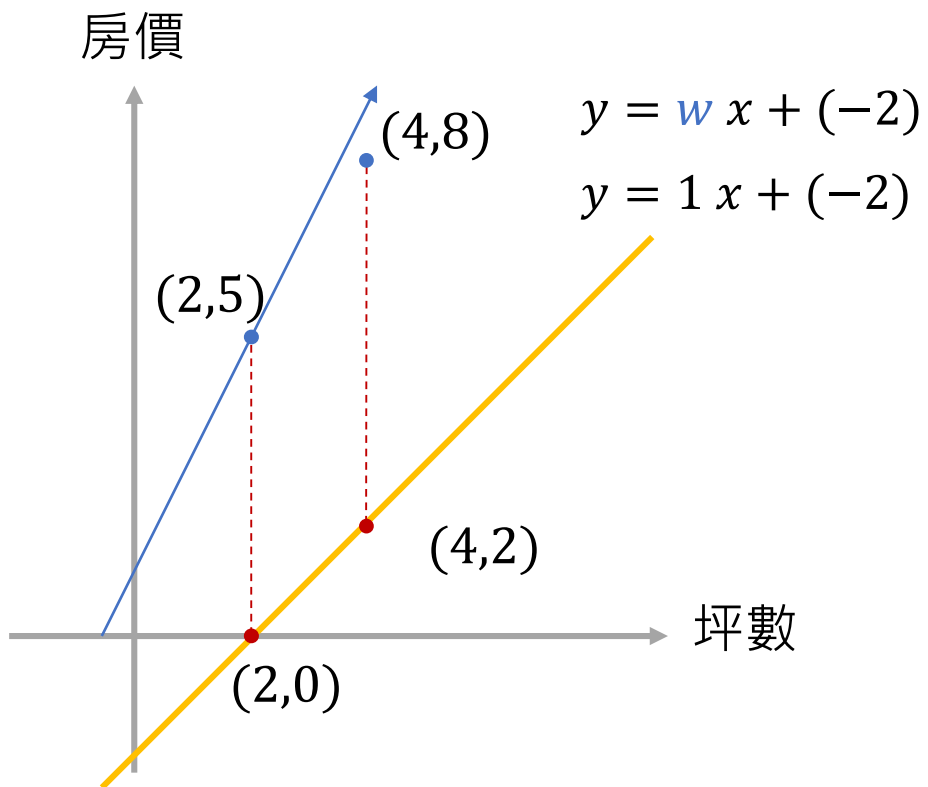
$$\text{Loss} = \frac{(5 - 0)^2 + (8 - 2)^2}{2} = 30.5$$

- 暫時固定 $b = -2$ ，考慮如何調整 w ，使 Loss 降低

$$\begin{aligned}\text{Loss} &= \frac{(5 - (2w - 2))^2 + (8 - (4w - 2))^2}{2} \\ &= 10w^2 - 54w + 74.5\end{aligned}$$

Loss Landscape 與最佳化困難

損失函數最低點在哪？

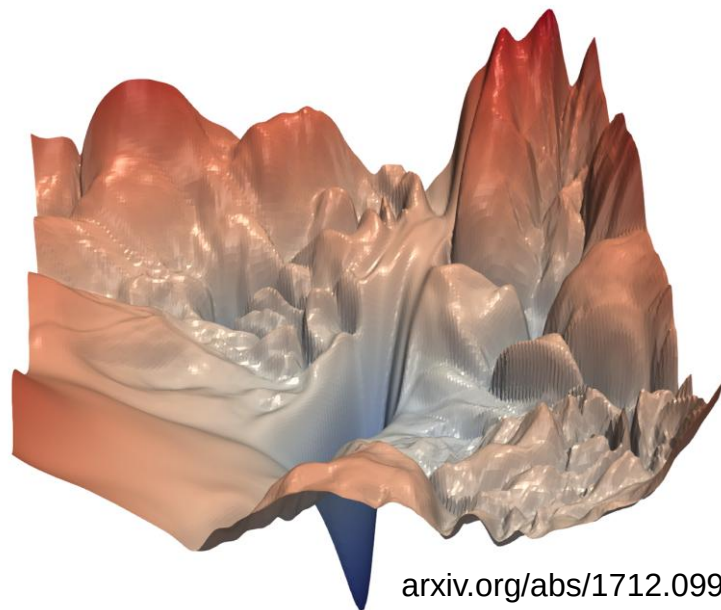


- 暫時固定 $b = -2$ ，考慮如何調整 w ，使 Loss 降低

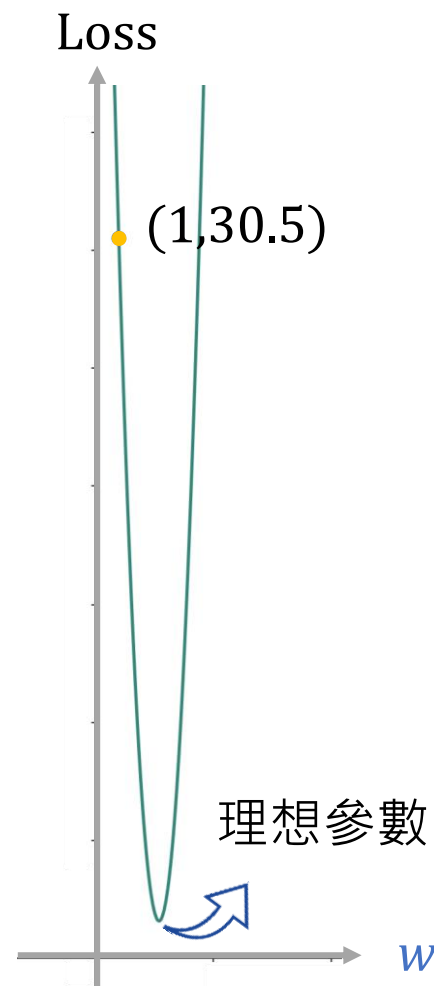
$$\text{Loss} = 10w^2 - 54w + 74.5$$

現實的殘酷

- 深度學習模型極度複雜，無法求解



- 參數 b 需同時調整

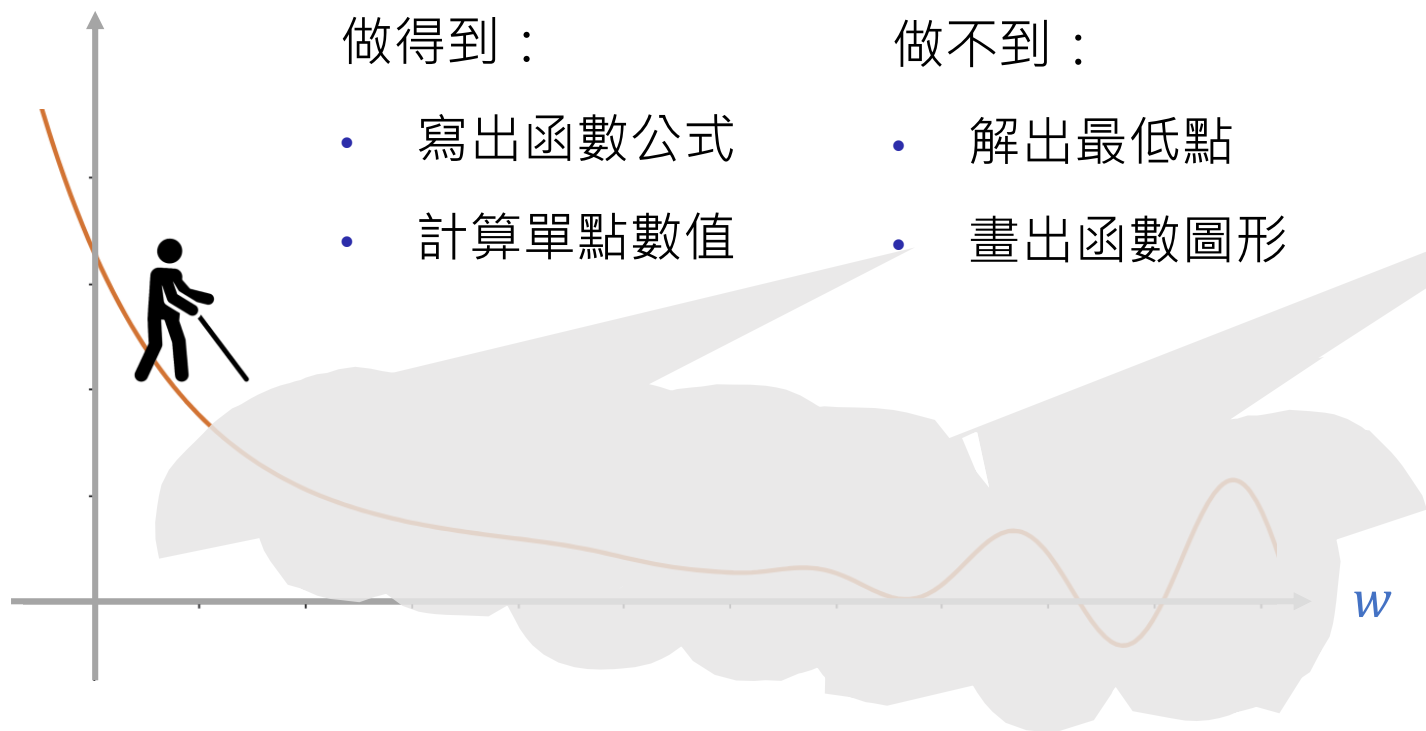


梯度下降概念

現實的殘酷

- 深度學習模型極度複雜，無法求解
- 參數 b 需同時調整

Loss



情境

被空投到濃霧瀰漫的山上，目標是下山

策略：梯度下降 (Gradient Descent)

- 感受腳下坡度 (**梯度**)
- 朝最陡的下坡方向，走一小步
- 重新感受腳下坡度、再走一步

特性

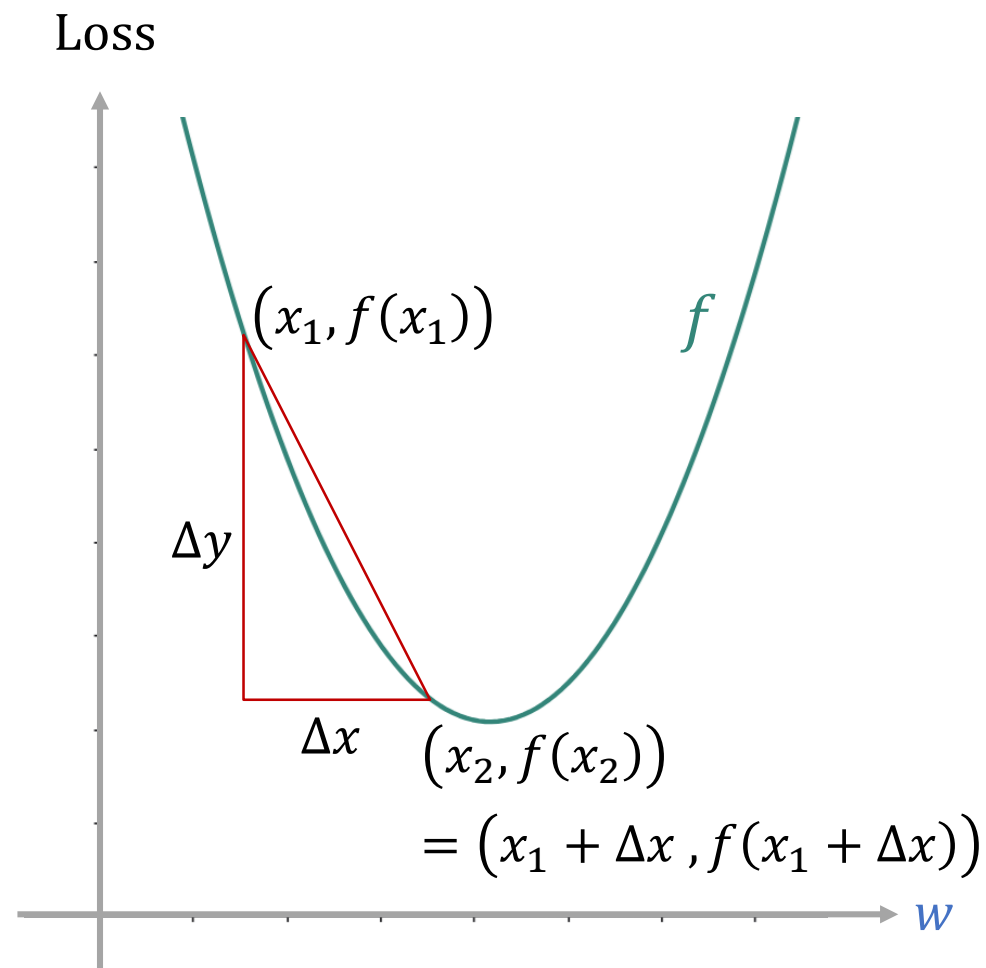
- 無法確保終點是最低點
- 可同時更新所有參數
(不同參數代表不同維度)

平均變化率

如何判斷坡度？

- 坡度 = 變化率

$$\begin{aligned}\text{平均坡度} &= \frac{\text{垂直變化率}}{\text{水平變化率}} = \frac{\Delta y}{\Delta x} = \frac{f(x_2) - f(x_1)}{x_2 - x_1} \\ &= \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}\end{aligned}$$



導數與瞬間變化率

如何判斷腳下精準坡度？

導數的直觀意義：稍微改動變量，對結果有什麼影響？

$$\text{平均坡度} = \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}$$

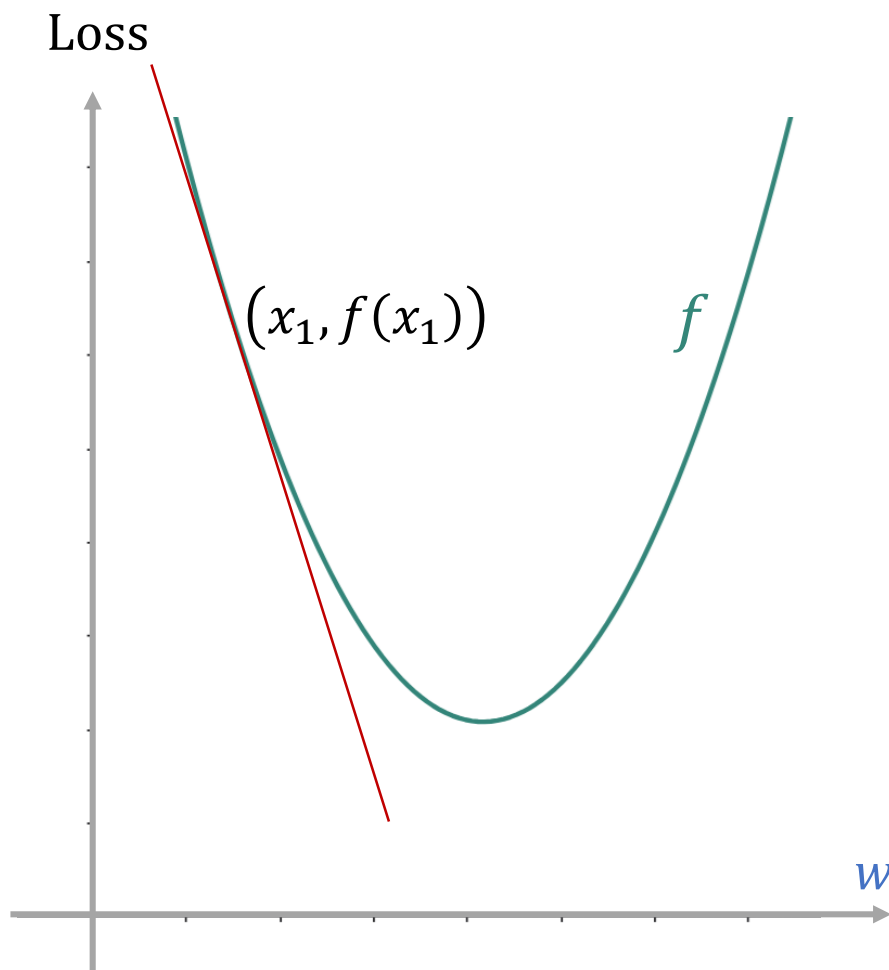
$$\text{腳下的精準坡度} = \text{瞬間變化率} = \lim_{\Delta x \rightarrow 0} \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}$$

= f 在 x_1 的切線斜率

= f 在 x_1 的**導數 (Derivative)**

$$= f'(x_1) = \lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x}(x_1) = \frac{dy}{dx}(x_1)$$

f' : f 上的點 $x \mapsto f$ 在 x_1 的切線斜率 (導數)
(f 的導函數)



線性函數的微分

導數 (Derivative)

$$f'(x_1) = f \text{ 在 } x_1 \text{ 的導數} = f \text{ 在 } x_1 \text{ 的切線斜率} = \lim_{\Delta x \rightarrow 0} \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}$$

- 線性函數

$$f \text{ 在 } x = 1 \text{ 的切線斜率} = f'(1)$$

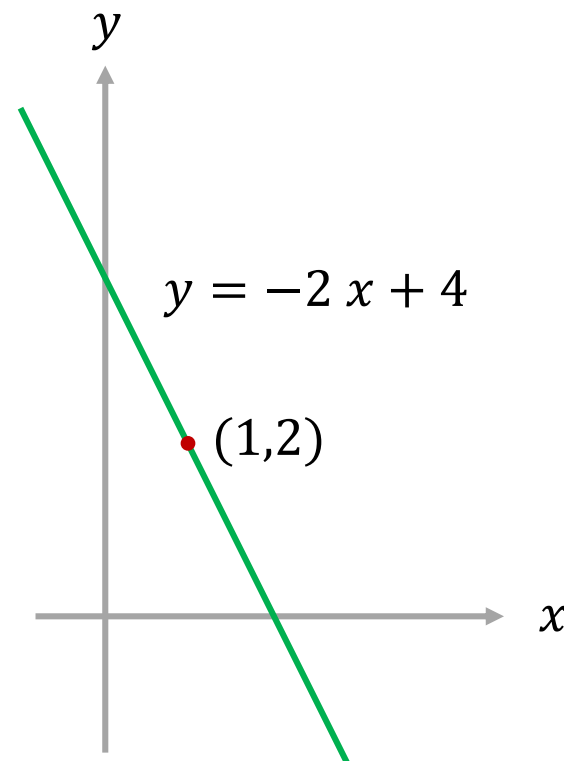
$$= \lim_{\Delta x \rightarrow 0} \frac{f(1 + \Delta x) - f(1)}{\Delta x}$$

$$= \lim_{\Delta x \rightarrow 0} \frac{[-2(1 + \Delta x) + 4] - [-2 \cdot 1 + 4]}{\Delta x}$$

$$= \lim_{\Delta x \rightarrow 0} \frac{-2\Delta x}{\Delta x} = \lim_{\Delta x \rightarrow 0} -2 = -2$$

$$f(x) = ax + b$$

$$f'(x) = a$$



二次函數的微分

導數 (Derivative)

$$f'(x_1) = f \text{ 在 } x_1 \text{ 的導數} = f \text{ 在 } x_1 \text{ 的切線斜率} = \lim_{\Delta x \rightarrow 0} \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}$$

- 二次函數

$$f \text{ 在 } x = 1 \text{ 的切線斜率} = f'(1)$$

$$= \lim_{\Delta x \rightarrow 0} \frac{f(1 + \Delta x) - f(1)}{\Delta x}$$

$$= \lim_{\Delta x \rightarrow 0} \frac{[2((1 + \Delta x) - 2)^2 + 1] - [2(1 - 2)^2 + 1]}{\Delta x}$$

$$= \lim_{\Delta x \rightarrow 0} \frac{2(\Delta x)^2 - 4\Delta x}{\Delta x} = \lim_{\Delta x \rightarrow 0} 2\Delta x - 4 = -4$$

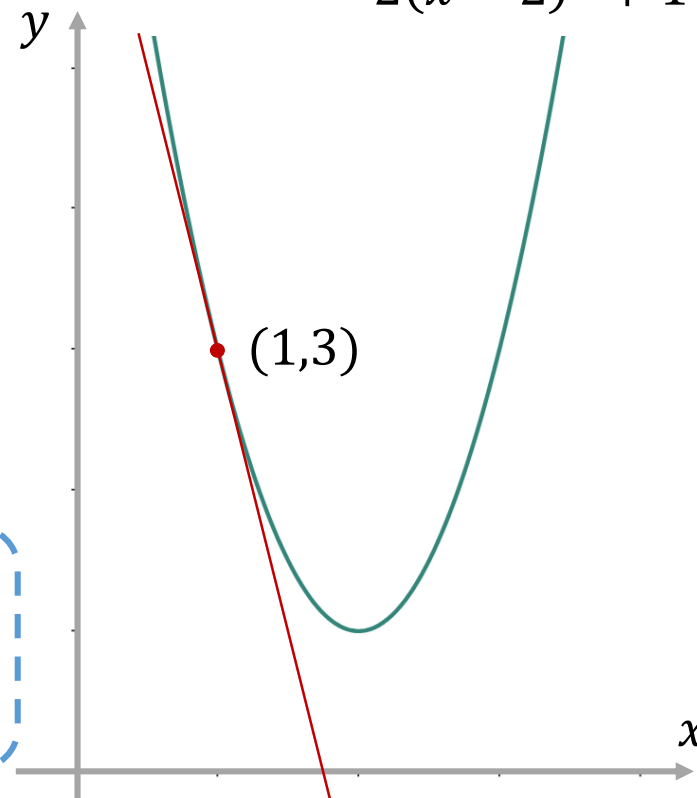
$$f(x) = ax^2 + bx + c$$

$$f'(x) = 2ax + b$$

$$f'(x) = 2 \cdot 2x - 8$$

$$f'(1) = 4 - 8$$

$$\begin{aligned} y &= 2x^2 - 8x + 9 \\ &= 2(x - 2)^2 + 1 \end{aligned}$$



多項式函數的微分

導數 (Derivative)

$$f'(x_1) = f \text{ 在 } x_1 \text{ 的導數} = f \text{ 在 } x_1 \text{ 的切線斜率} = \lim_{\Delta x \rightarrow 0} \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}$$

- 多項式函數

$$f(x) = ax^2 + bx + c$$

$$f'(x) = 2ax + b$$

$$f(x) = x^n$$

$$f'(x) = nx^{n-1}$$

$$f(x) = 2x^5 - 4x^3 + 6x$$

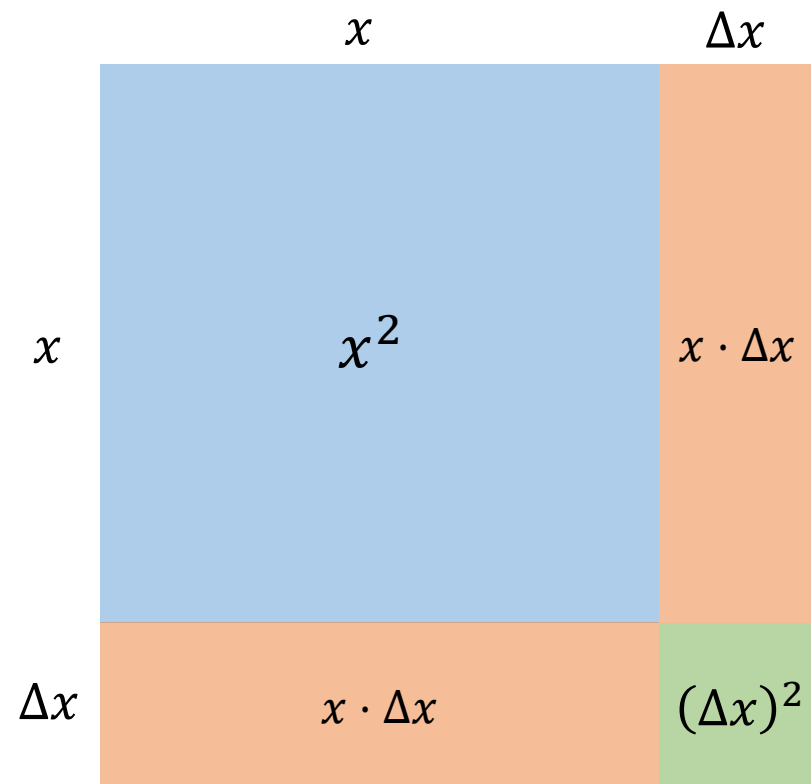
$$f'(x) = 10x^4 - 12x^2 + 6$$

$$f(x) = x^2$$

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{(x + \Delta x)^2 - x^2}{\Delta x}$$

$$= \lim_{\Delta x \rightarrow 0} \frac{2x \cdot \Delta x + (\Delta x)^2}{\Delta x}$$

$$= \lim_{\Delta x \rightarrow 0} 2x + \Delta x = 2x$$



ReLU 的微分

導數 (Derivative)

$$f'(x_1) = f \text{ 在 } x_1 \text{ 的導數} = f \text{ 在 } x_1 \text{ 的切線斜率} = \lim_{\Delta x \rightarrow 0} \frac{f(x_1 + \Delta x) - f(x_1)}{\Delta x}$$

- ReLU

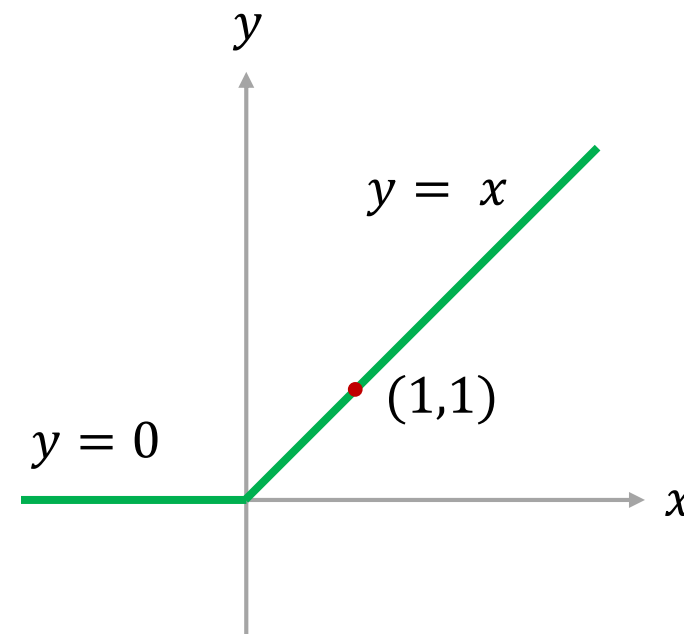
$$f(x) = \max(0, x)$$

$$f'(x) = \begin{cases} 0, & \text{if } x < 0 \\ 1, & \text{if } x > 0 \end{cases}$$

在 $x = 0$: 不可微 (導數無定義)

平滑曲線才可微分

- 此情況幾乎不會發生
- 程式碼直接定義導數為 0

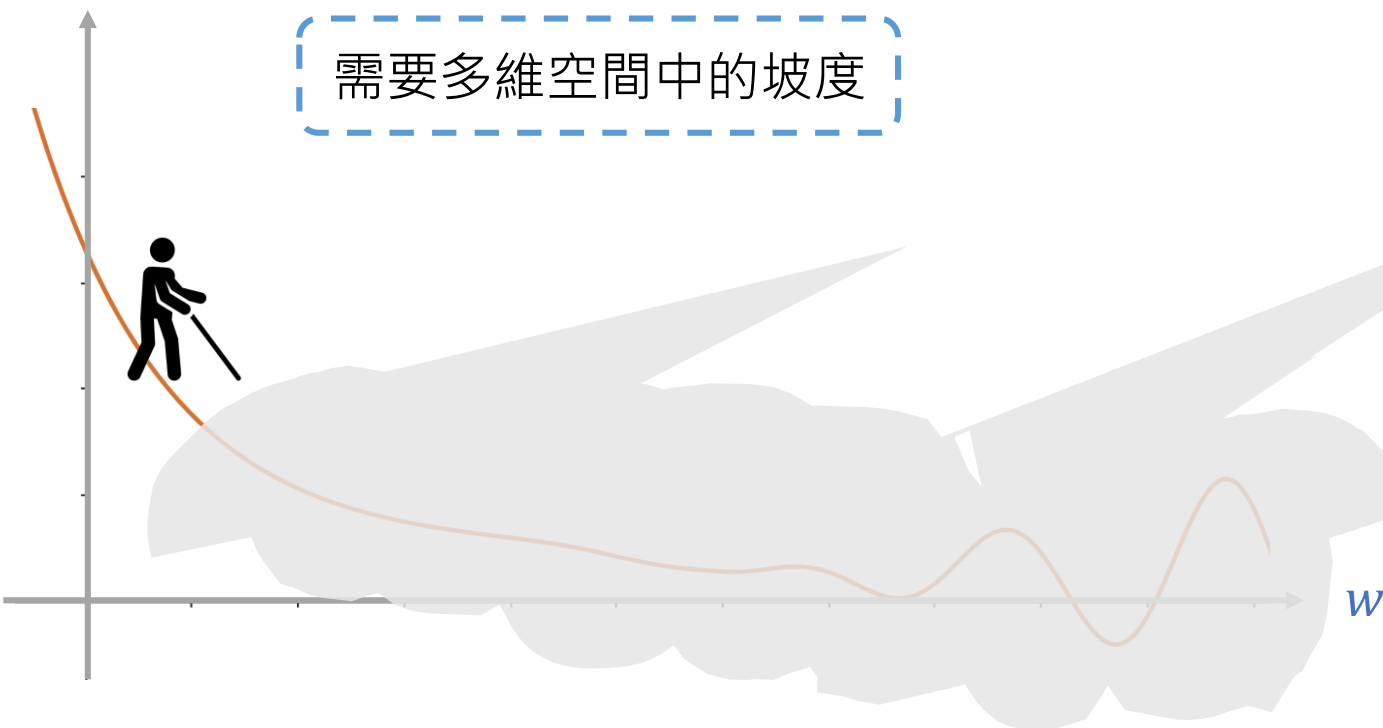


多變數最佳化問題

所有參數都要更新

- 情境：被空投到濃霧瀰漫的山上，目標是下山
- 已知如何判斷 w 移動方向
- 模型有多個參數需同時調整

需要多維空間中的坡度



策略：梯度下降 (Gradient Descent)

- 感受腳下坡度 (**梯度**)
- 朝最陡的下坡方向，走一小步
- 重新感受腳下坡度、再走一步

特性

- 無法確保終點是最低點
- 可同時更新所有參數
(不同參數代表不同維度)

偏微分

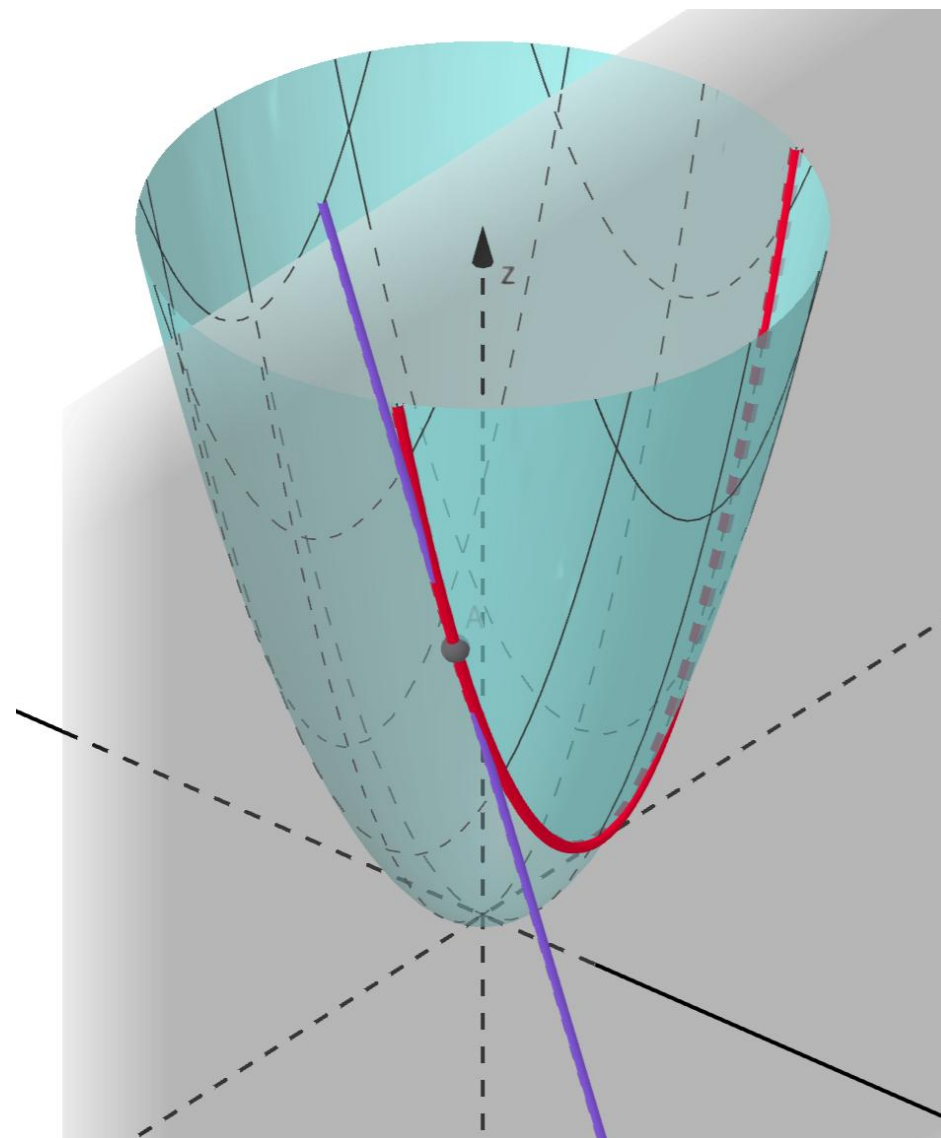
一次只看一個參數

把其他所有變數視為常數，只對其中一個變數求導數

- 如同先前假設 b 固定，損失函數變成只有單變數 w
- 計算損失函數對 w 的偏微分 $\frac{\partial L}{\partial w}$
- 改成固定 w ，計算損失函數對 b 的偏微分 $\frac{\partial L}{\partial b}$

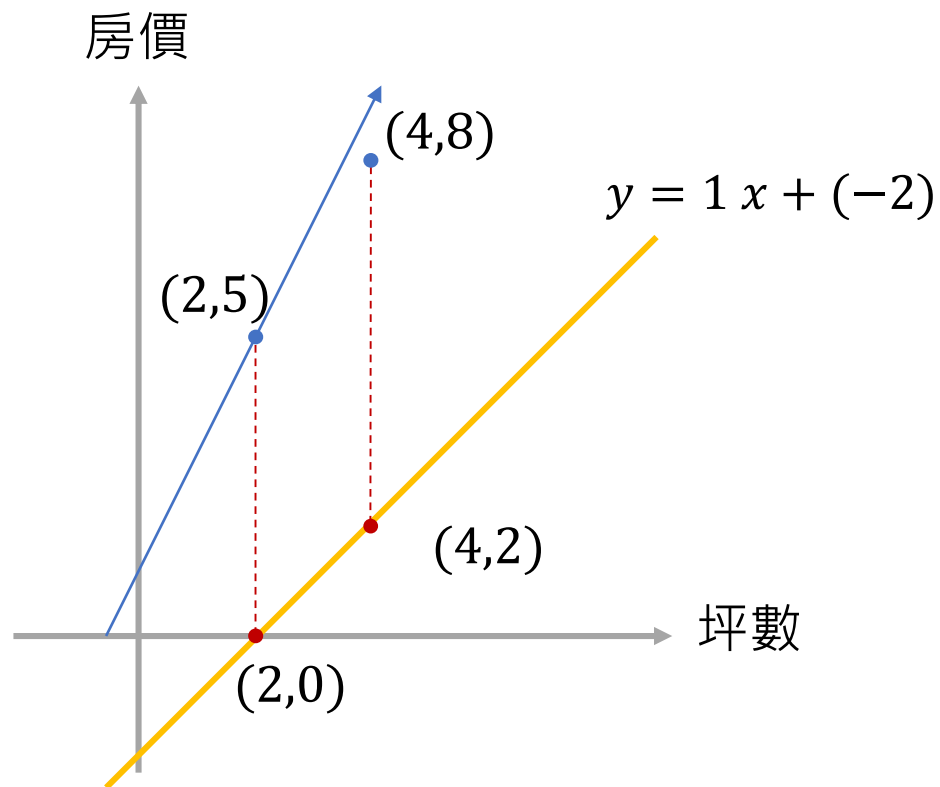
$$L(w, b) = w^3 + 7b + 2wb$$

$$\frac{\partial L}{\partial w} = 3w^2 + 2b, \quad \frac{\partial L}{\partial b} = 7 + 2w$$



Loss 對模型參數的偏微分

算出每個參數的坡度



- 真實的 Loss 有 w, b 兩個變數

$$\text{Loss} = \frac{(5 - (2w + b))^2 + (8 - (4w + b))^2}{2}$$
$$= 10w^2 + b^2 + 6wb - 42w - 13b + 44.5$$

$$\frac{\partial L}{\partial w} = 20w + 6b - 42 = -34$$

w 應調大

代入隨機初始化的點 $(w, b) = (1, -2)$

$$\frac{\partial L}{\partial b} = 2b + 6w - 13 = -11$$

b 應調大

梯度

把所有坡度集合起來

梯度：由所有偏微分組成的向量 $\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$
(Gradient)

- Loss = $10w^2 + b^2 + 6wb - 42w - 13b + 44.5$

$$\frac{\partial L}{\partial w}(1,2) = -34$$

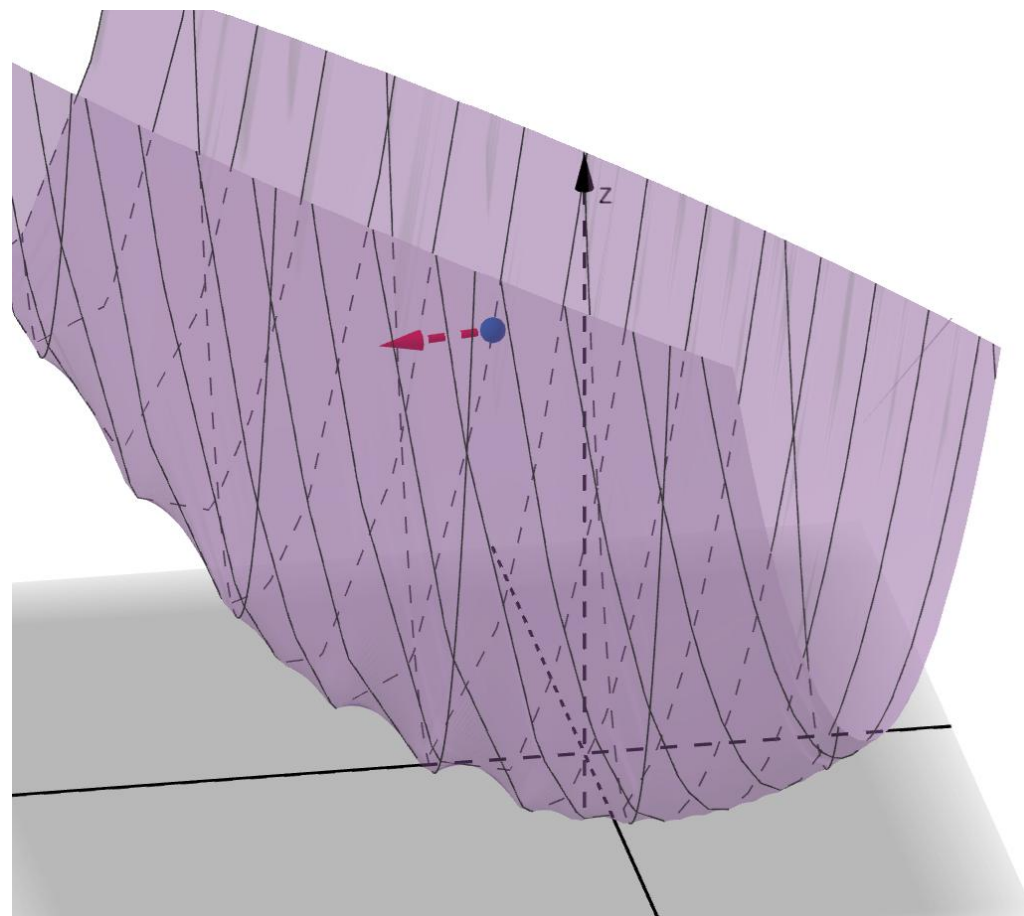
w 應調大

$$\frac{\partial L}{\partial b}(1,2) = -11$$

b 應調大

$$\nabla L(1,2) = \begin{bmatrix} -34 \\ -11 \end{bmatrix}$$

w 和 b 應同時調大，且調整幅度為 34: 11



梯度的方向

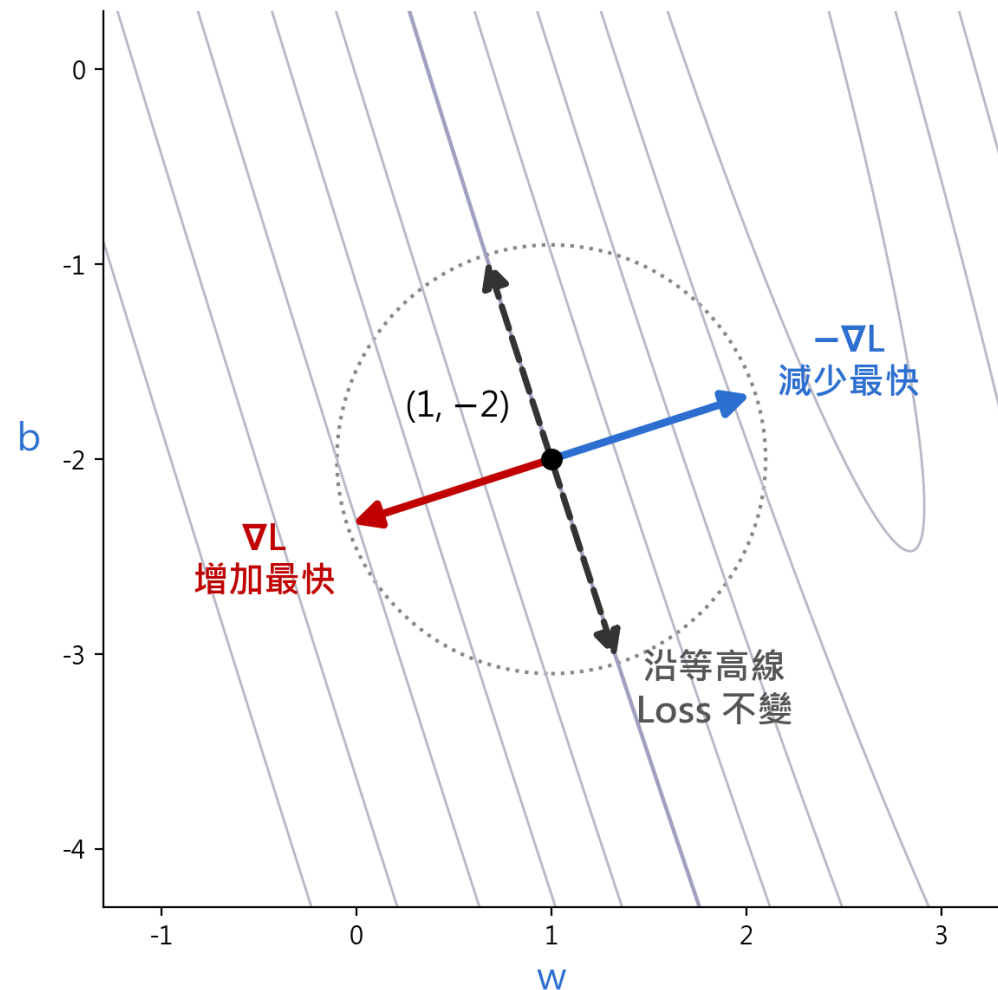
- 往任意方向走一小步 $(\Delta w, \Delta b)$ ，Loss 的變化量：

$$\Delta L \approx \frac{\partial L}{\partial w} \Delta w + \frac{\partial L}{\partial b} \Delta b = \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix} \cdot \begin{bmatrix} \Delta w \\ \Delta b \end{bmatrix} = \nabla L \cdot \begin{bmatrix} \Delta w \\ \Delta b \end{bmatrix}$$
$$= \|\nabla L\| \times \text{步長} \times \cos \theta \quad (\theta : \text{步伐與 } \nabla L \text{ 的夾角})$$

- $\theta = 0^\circ$ (沿 ∇L) $\cos \theta = 1 \rightarrow$ **Loss 增加最快**
- $\theta = 180^\circ$ (沿 $-\nabla L$) $\cos \theta = -1 \rightarrow$ **Loss 減少最快**

梯度 ∇L 指向函數**增長最快**的方向

梯度下降就往反方向 $-\nabla L$ 走：最陡的下坡

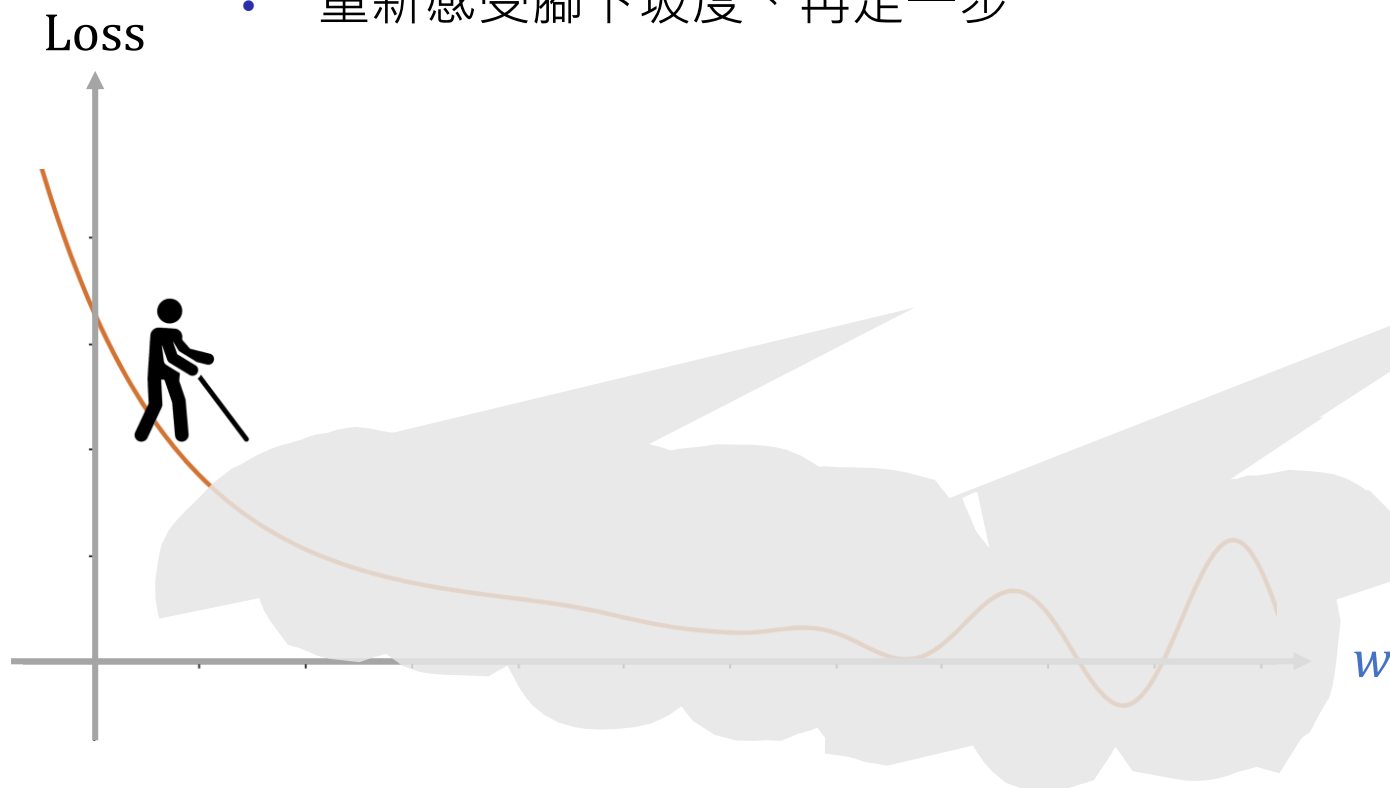


虛線圓：步長相同的所有方向；灰線：Loss 等高線

梯度下降 - 參數更新與學習率

策略：梯度下降 (Gradient Descent)

- 感受腳下坡度 (**梯度**) ✓
- 朝最陡的下坡方向，走一小步
- 重新感受腳下坡度、再走一步



- 新位置 = 舊位置 - 步伐大小 × 下山方向

$$\begin{bmatrix} w_{\text{新}} \\ b_{\text{新}} \end{bmatrix} = \begin{bmatrix} w_{\text{舊}} \\ b_{\text{舊}} \end{bmatrix} - \eta \times \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ -2 \end{bmatrix} - 0.05 \times \begin{bmatrix} -34 \\ -11 \end{bmatrix} = \begin{bmatrix} 2.7 \\ -1.45 \end{bmatrix}$$

η = 學習率 (Learning Rate)

- 太大可能一步跨過山谷，甚至發散
- 太小則速度極慢

梯度下降 - 結果驗證

新位置 = 舊位置 - 步伐大小 $\times$ 下山方向

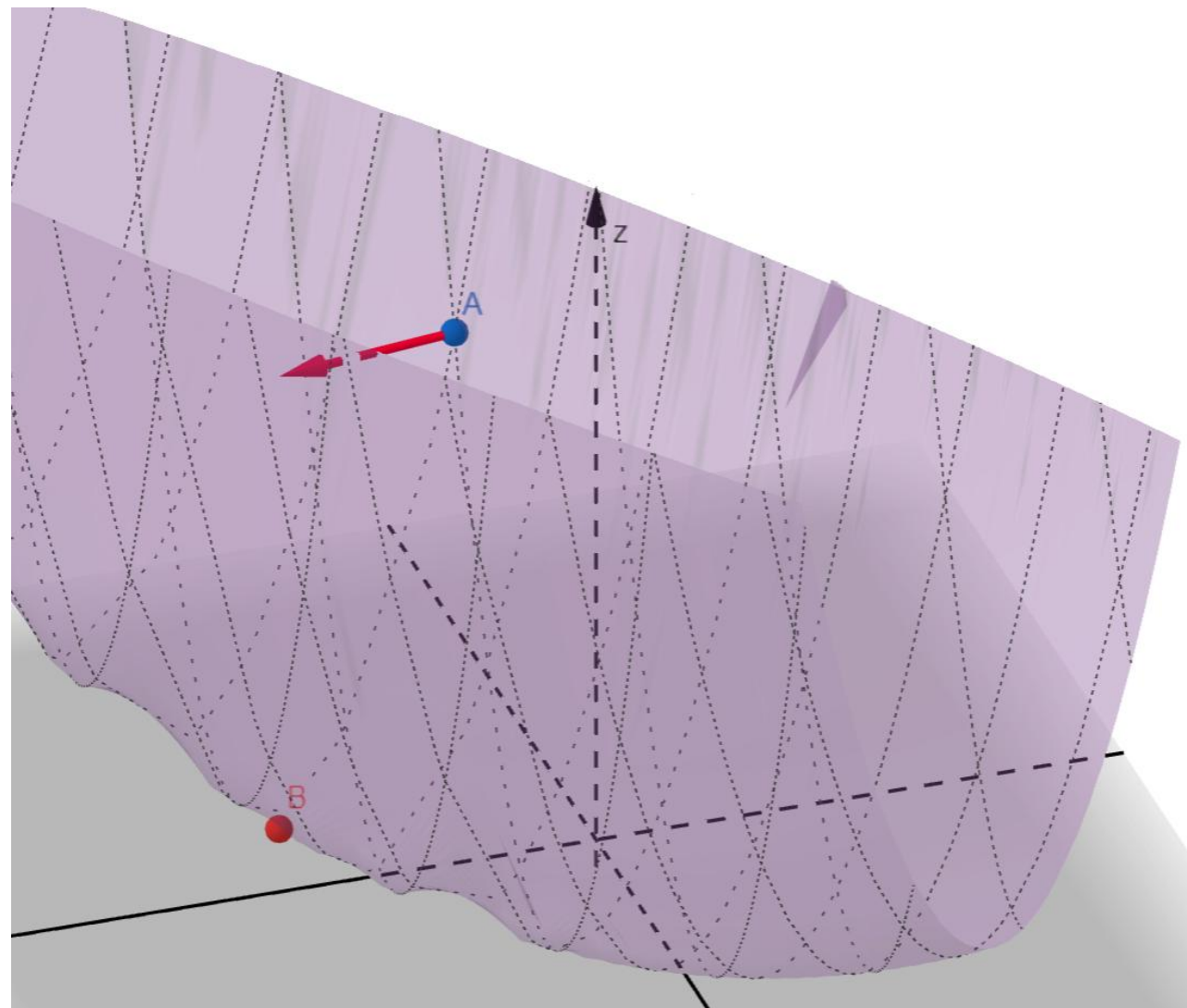
$$\begin{bmatrix} w_{\text{新}} \\ b_{\text{新}} \end{bmatrix} = \begin{bmatrix} w_{\text{舊}} \\ b_{\text{舊}} \end{bmatrix} - \eta \times \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ -2 \end{bmatrix} - 0.05 \times \begin{bmatrix} -34 \\ -11 \end{bmatrix} = \begin{bmatrix} 2.7 \\ -1.45 \end{bmatrix}$$

- $\text{Loss} = 10w^2 + b^2 + 6wb - 42w - 13b + 44.5$

$$\text{Loss}(1, -2) = 30.5$$

$$\text{Loss}(2.7, -1.45) = 1.4625$$



梯度下降 - 結果驗證

新位置 = 舊位置 - 步伐大小 × 下山方向

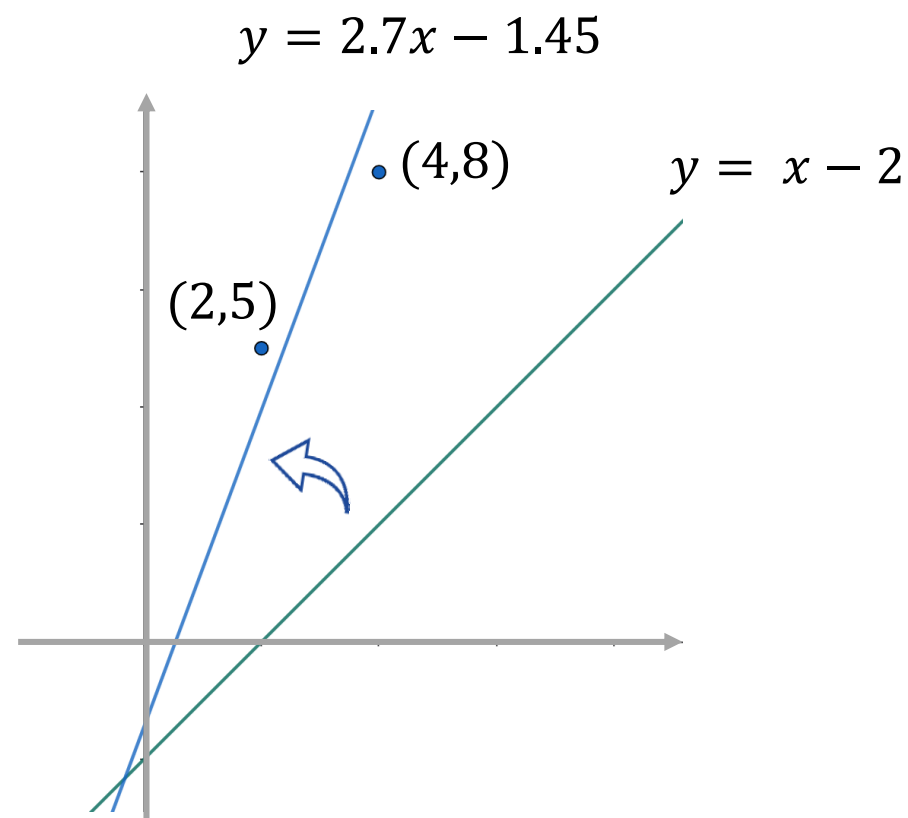
$$\begin{bmatrix} w_{\text{新}} \\ b_{\text{新}} \end{bmatrix} = \begin{bmatrix} w_{\text{舊}} \\ b_{\text{舊}} \end{bmatrix} - \eta \times \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ -2 \end{bmatrix} - 0.05 \times \begin{bmatrix} -34 \\ -11 \end{bmatrix} = \begin{bmatrix} 2.7 \\ -1.45 \end{bmatrix}$$

- Loss = $10w^2 + b^2 + 6wb - 42w - 13b + 44.5$

$$\text{Loss}(1, -2) = 30.5$$

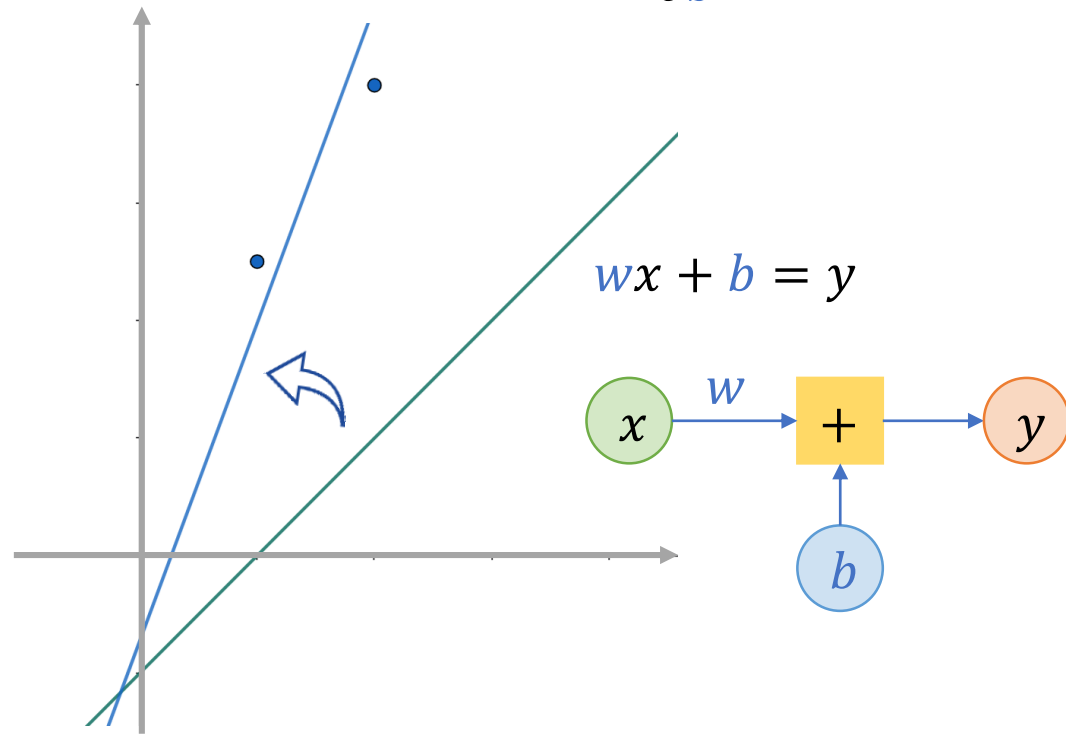
$$\text{Loss}(2.7, -1.45) = 1.4625$$



深度神經網路與梯度計算

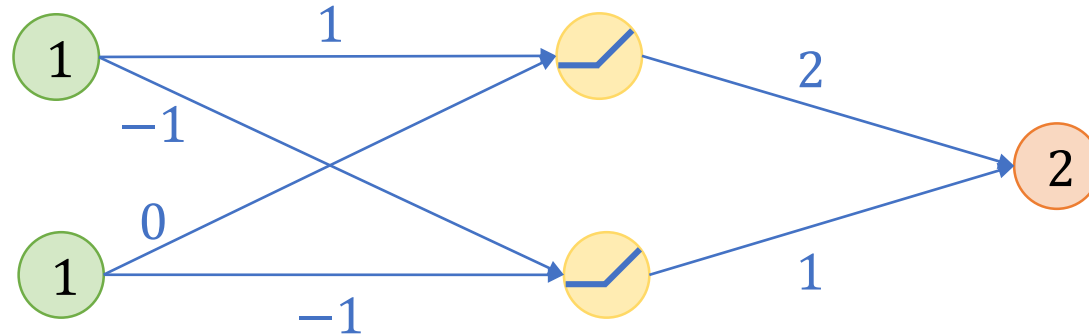
新位置 = 舊位置 - 步伐大小 $\times$ 下山方向

$$\begin{bmatrix} w_{\text{新}} \\ b_{\text{新}} \end{bmatrix} = \begin{bmatrix} w_{\text{舊}} \\ b_{\text{舊}} \end{bmatrix} - \eta \times \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$



- MLP (Multilayer Perceptron)

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix} \xrightarrow{\begin{bmatrix} 1 & 0 \\ -1 & -1 \end{bmatrix}} \begin{bmatrix} 1 \\ -2 \end{bmatrix} \xrightarrow{\text{ReLU}} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \xrightarrow{\begin{bmatrix} 2 & 1 \end{bmatrix}} 2$$

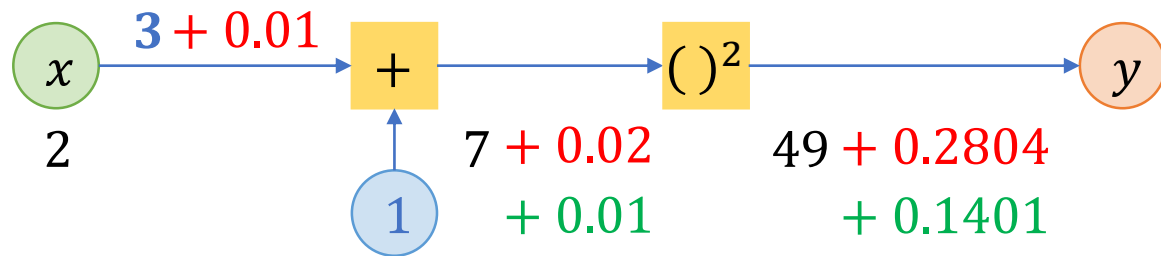


- 深度學習模型有多層函數合成

- 稍微改動前面的變量，經過層層轉換後，對結果有什麼影響？
- 在複雜的流程中，如何評估每個節點的責任

前向傳播與擾動傳遞

$$(wx + b)^2 = y$$



前向傳播
(Forward Pass)
 $2 \cdot 3 + 1 = 7$, $7^2 = 49$

- 深度學習模型有多層函數合成
 - 稍微改動前面的變量，經過層層轉換後，對結果有什麼影響？
 - 在複雜的流程中，如何評估每個節點的責任

- 視為一個複雜函數

$$\frac{\Delta y}{\Delta w} = \frac{0.2804}{0.01} \xrightarrow{\Delta w \rightarrow 0} \frac{dy}{dw} = 28$$

- 視為兩個函數合成 $x \rightarrow u \rightarrow y$

$$wx + b = u, \quad u^2 = y$$

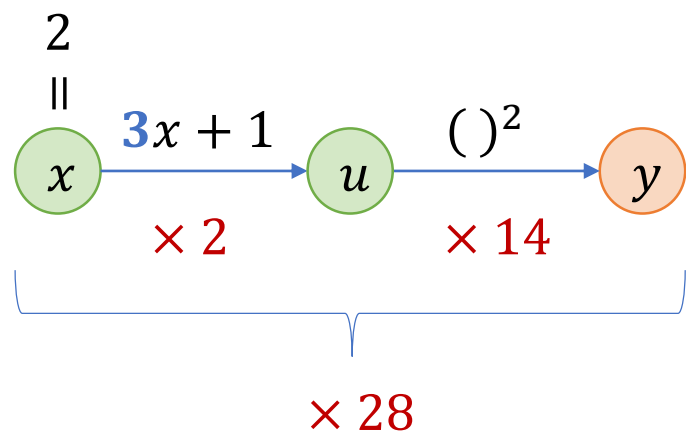
$$\frac{\Delta u}{\Delta w} = \frac{0.02}{0.01} \xrightarrow{\Delta w \rightarrow 0} \frac{du}{dw} = 2$$

$$\frac{\Delta y}{\Delta u} = \frac{0.1401}{0.01} \xrightarrow{\Delta u \rightarrow 0} \frac{dy}{du} = 14$$

$$\Rightarrow \frac{dy}{dw} = \frac{dy}{du} \times \frac{du}{dw} = 14 \times 2$$

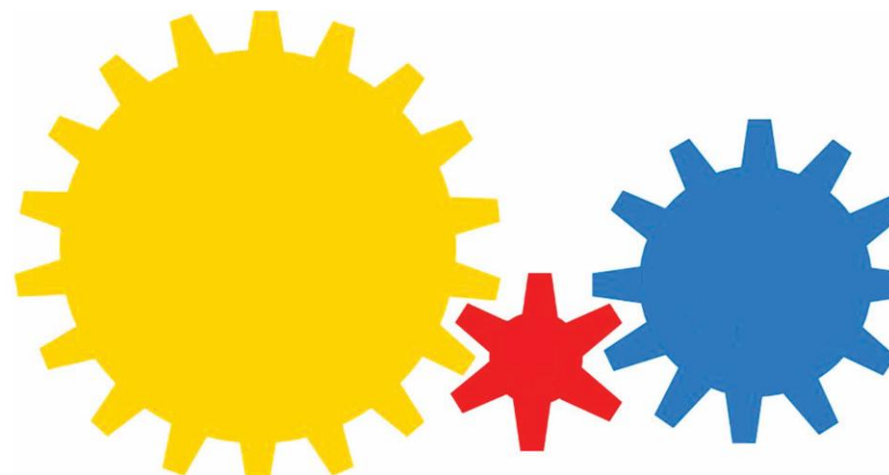
連鎖律 (Chain Rule)

- 整體影響 = 局部影響相乘



連鎖律 (Chain Rule)

$$\frac{dy}{dw} = \frac{dy}{du} \times \frac{du}{dw}$$



轉 1 圈

轉 3 圈

轉 1 圈

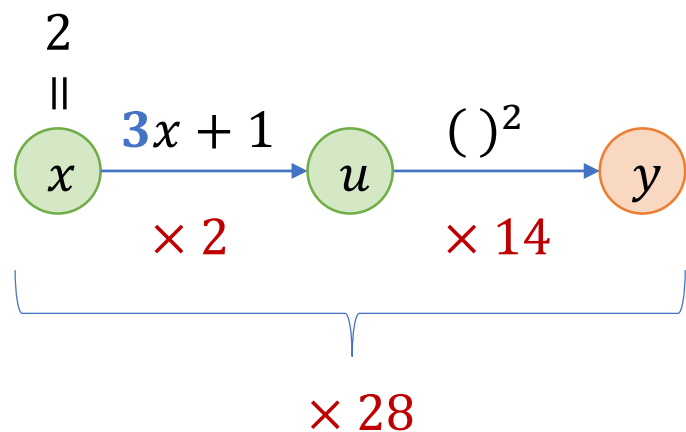
轉 0.5 圈

轉 1 圈

轉 1.5 圈

多變數連鎖律

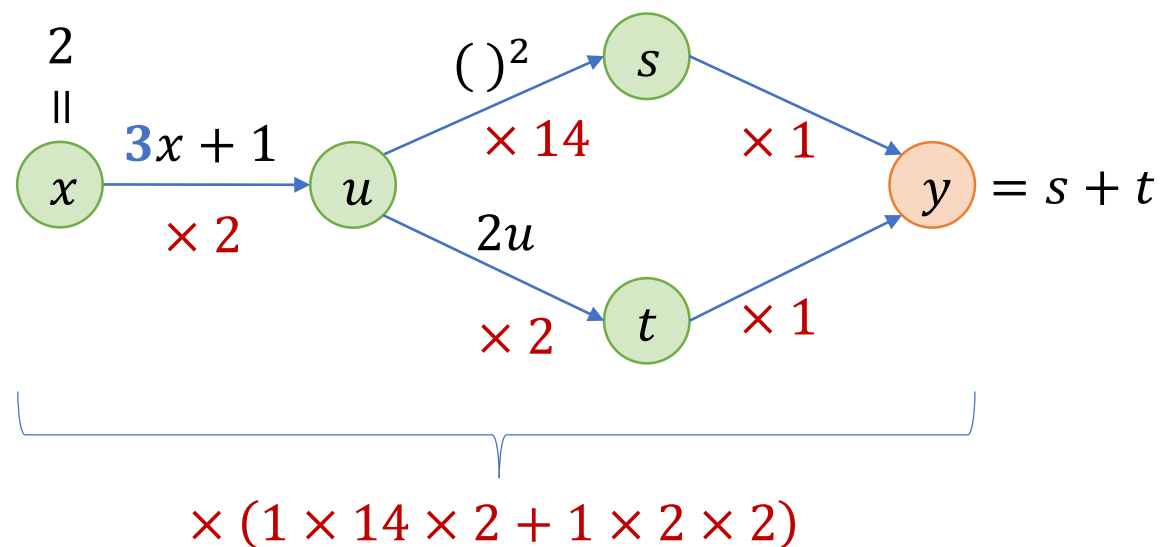
- 整體影響 = 局部影響相乘



連鎖律 (Chain Rule)

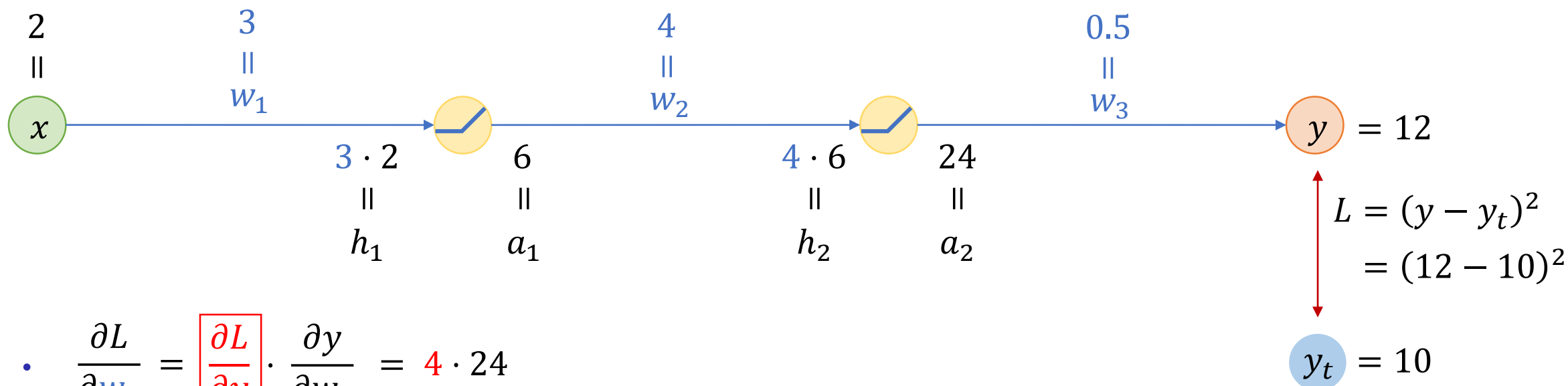
$$\frac{dy}{dw} = \frac{dy}{du} \times \frac{du}{dw}$$

- 串聯相乘，並聯相加



$$\begin{aligned} \frac{dy}{dw} &= \frac{\partial y}{\partial s} \times \frac{\partial s}{\partial u} \times \frac{\partial u}{\partial w} \\ &+ \frac{\partial y}{\partial t} \times \frac{\partial t}{\partial u} \times \frac{\partial u}{\partial w} \end{aligned}$$

Naïve 連鎖律效率分析



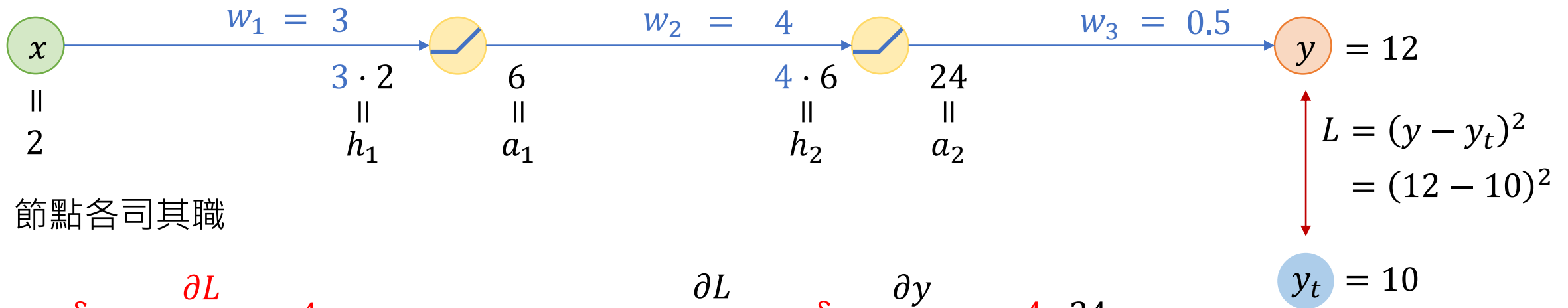
痛點

- 局部梯度重複計算多次
- 局部梯度相乘重複計算多次

解法：傳遞梯度

- 傳遞局部梯度相乘後結果

反向傳播 (Backpropagation)



● 節點各司其職

- $\delta_y = \frac{\partial L}{\partial y} = 4$

- $\delta_{h_2} = \delta_y \cdot \frac{\partial y}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_2} = 4 \cdot 0.5 \cdot 1$

- $\delta_{h_1} = \delta_{h_2} \cdot \frac{\partial h_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial h_1} = 2 \cdot 4 \cdot 1$

上游梯度

局部梯度

$$\frac{\partial L}{\partial w_3} = \delta_y \cdot \frac{\partial y}{\partial w_3} = 4 \cdot 24$$

$$\frac{\partial L}{\partial w_2} = \delta_{h_2} \cdot \frac{\partial h_2}{\partial w_2} = 2 \cdot 6$$

$$\frac{\partial L}{\partial w_1} = \delta_{h_1} \cdot \frac{\partial h_1}{\partial w_1} = 8 \cdot 2$$

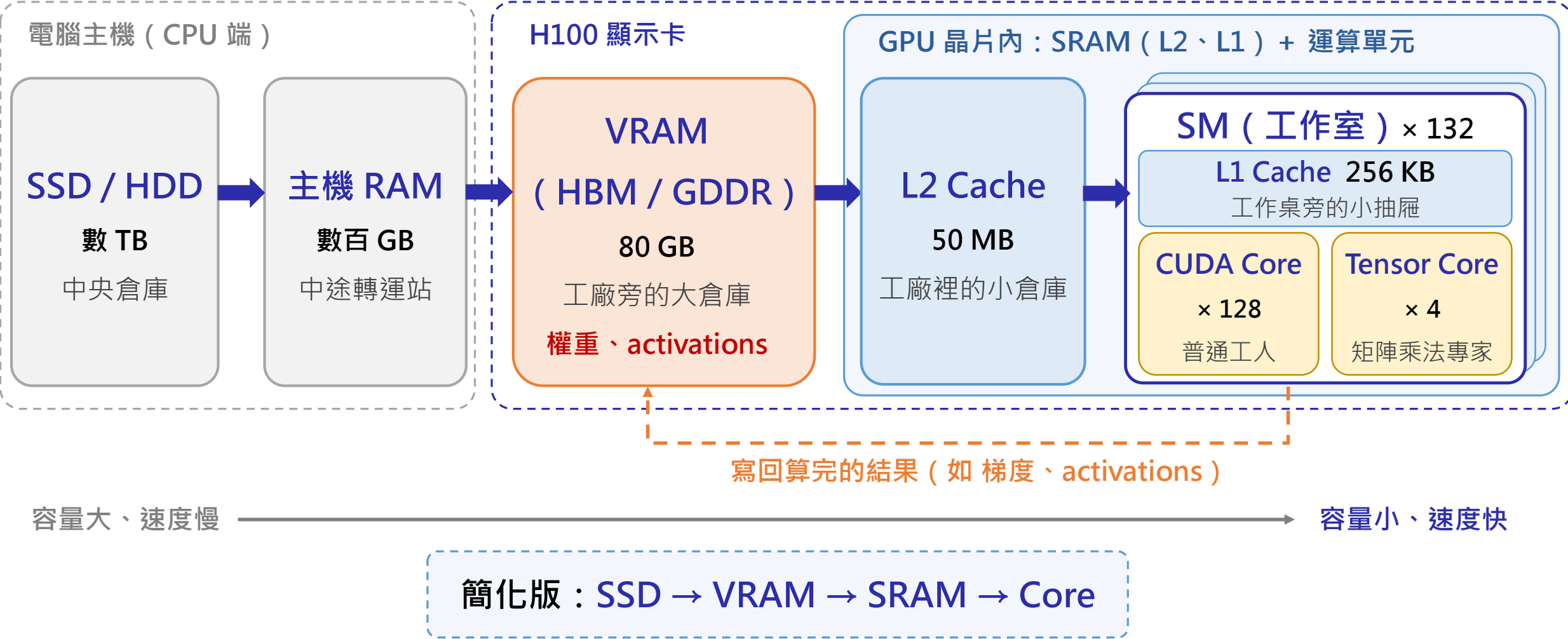
參數梯度

● 每層計算量相同

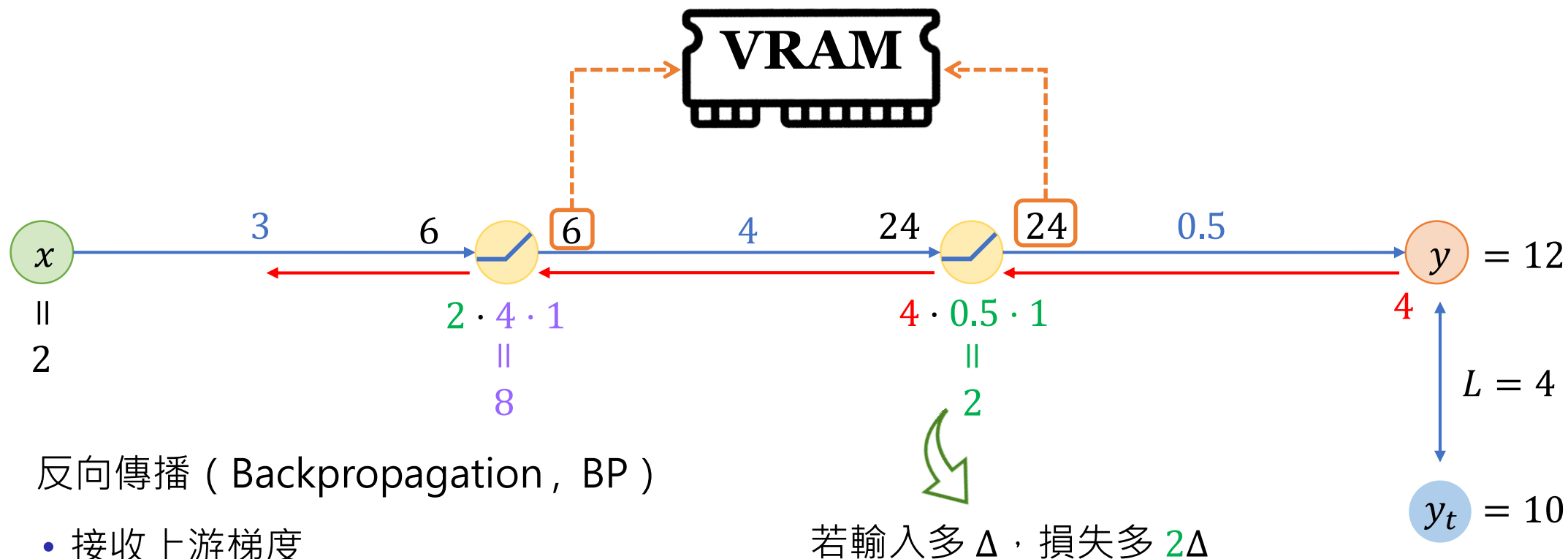
- 接收上游梯度
- 乘上局部梯度
- 傳遞給下游
- 計算自身參數梯度
- (不用知道來龍去脈)

GPU 記憶體階層

資料如何從硬碟送到運算單元 以 NVIDIA H100 為例



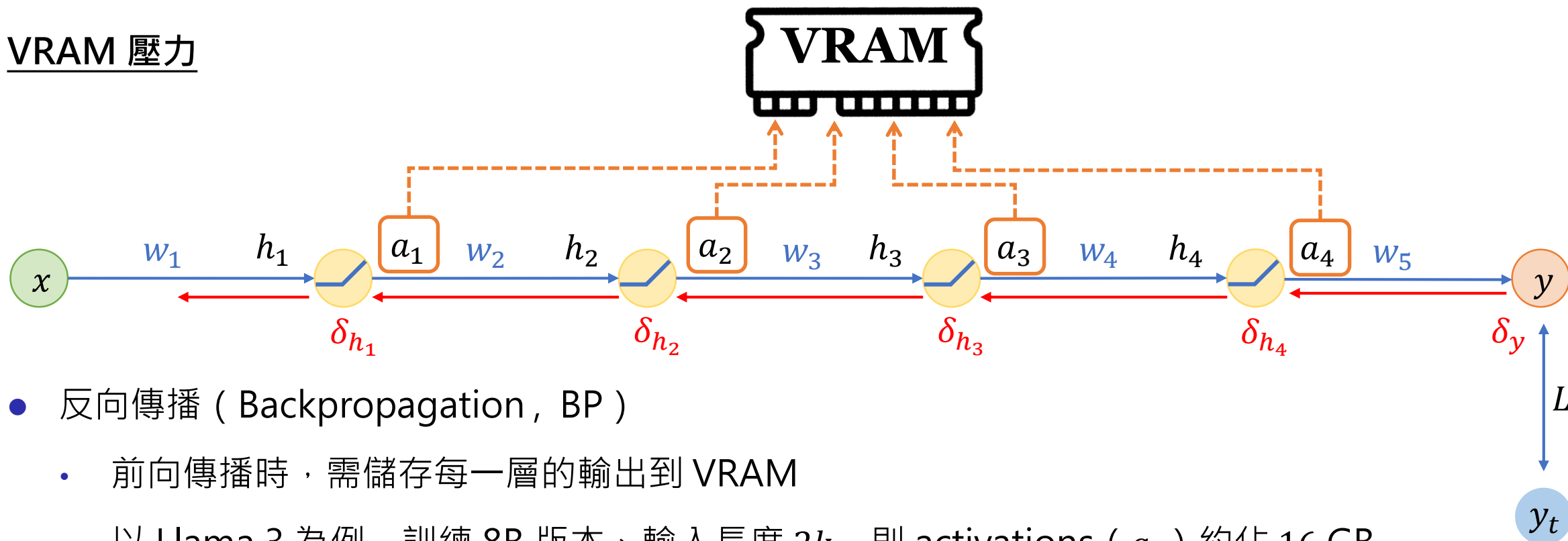
梯度計算資訊流與記憶體依賴性



- 反向傳播 (Backpropagation , BP)
 - 接收上游梯度
 - 乘上局部梯度
 - 傳遞給下游
 - 計算自身參數梯度 $\Rightarrow$ 需知道該層的輸入 $\Rightarrow$ 前向傳播時，需儲存每一層的輸出到 VRAM

記憶體瓶頸 - Activations 與 VRAM

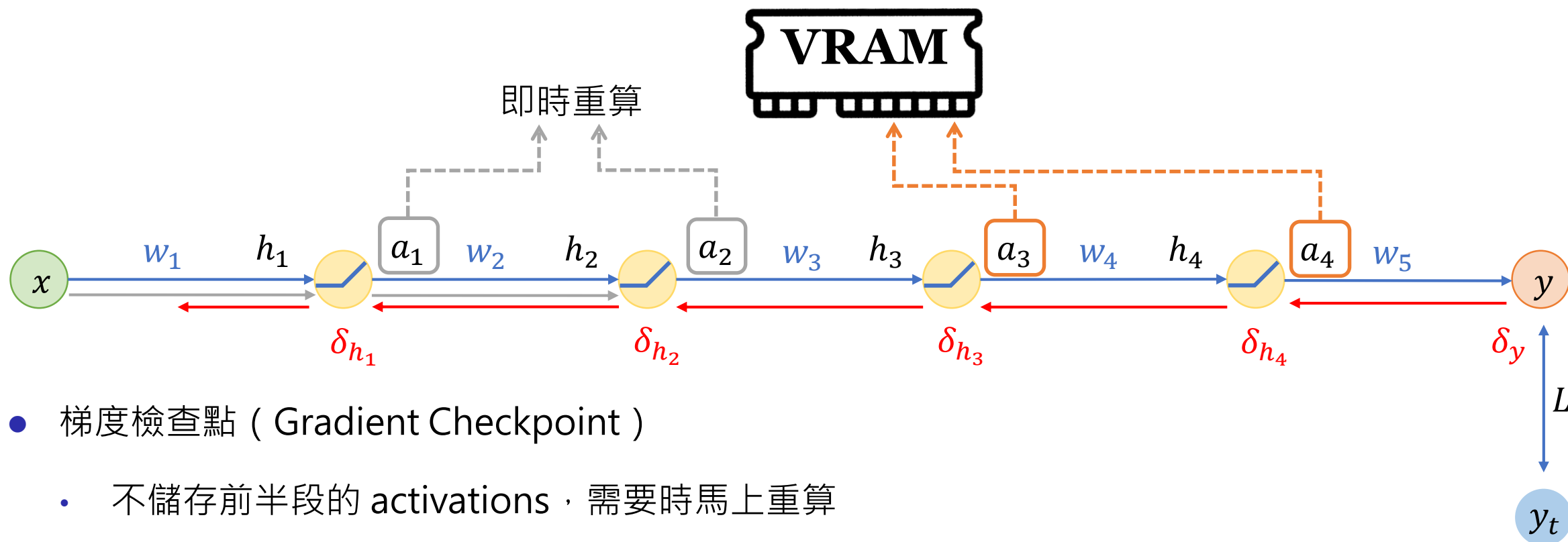
VRAM 壓力



- 反向傳播 (Backpropagation , BP)

- 前向傳播時，需儲存每一層的輸出到 VRAM
- 以 Llama 3 為例，訓練 8B 版本、輸入長度 $2k$ ，則 activations (a_i) 約佔 16 GB
 - activations 包含 attention matrix，此部分以 (輸入長度)² 成長
(使用 FlashAttention 可不儲存 attention matrix (即時重算)，儲存量降至約 8 GB)

梯度檢查點 (Gradient Checkpoint)



- 梯度檢查點 (Gradient Checkpoint)

- 不儲存前半段的 activations，需要時馬上重算
(前向傳播算完即丟) (重算的前向傳播不需走到最後)

- 以時間換空間 (trade-off)

耗時增加約 30%，VRAM 佔用 < 1 GB

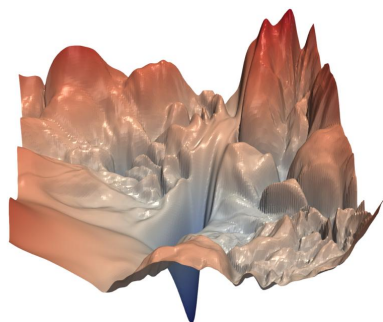
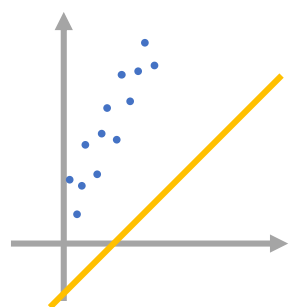
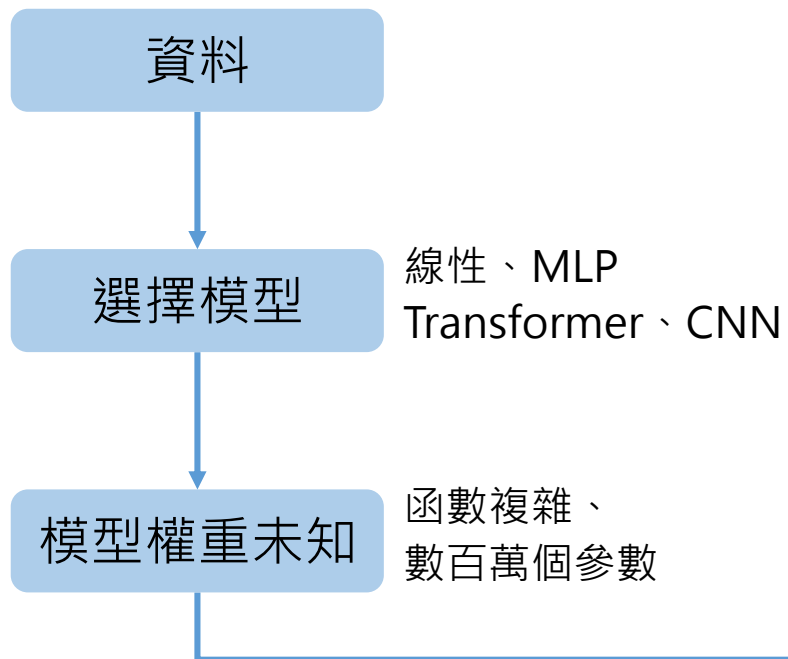
梯度檢查點 (Gradient Checkpoint)

- 梯度檢查點 (Gradient Checkpoint)

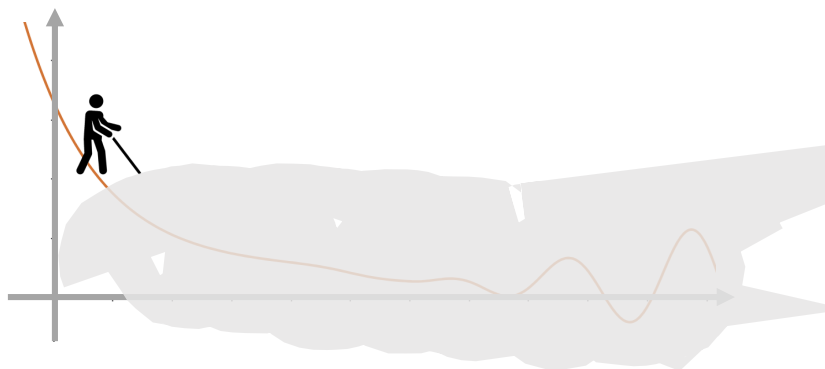
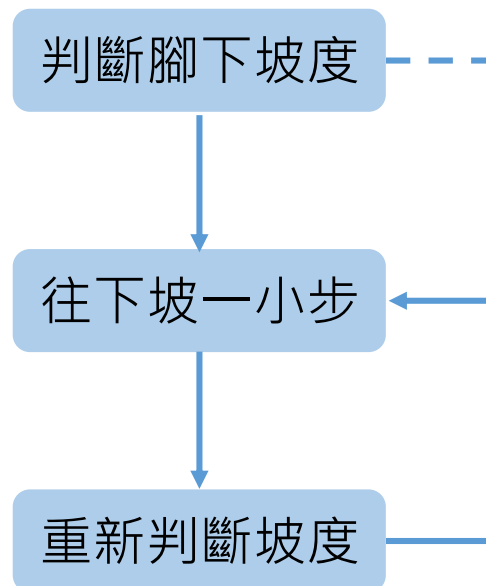
```
1 from transformers import TrainingArguments
2
3 training_args = TrainingArguments(
4     output_dir="./results",           # 儲存路徑
5     num_train_epochs=10,              # 訓練輪數
6     per_device_train_batch_size=4,    # 實體 Batch Size
7     gradient_accumulation_steps=4,    # 梯度累積步數
8     learning_rate=5e-5,              # 初始學習率
9     lr_scheduler_type="linear",       # 學習率排程器
10    optim="adamw_torch",              # AdamW 優化器
11    fp16=True,                        # 開啟 FP16 混合精度
12    gradient_checkpointing=True,       # 梯度檢查點
```

總結（一） - 模型最佳化與微分

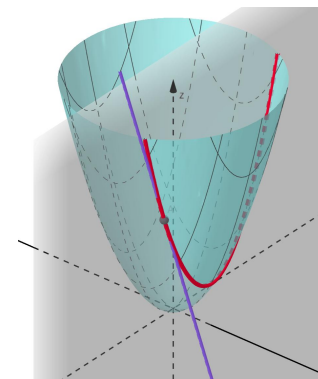
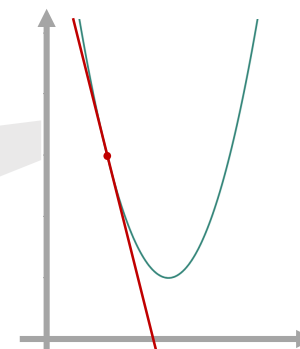
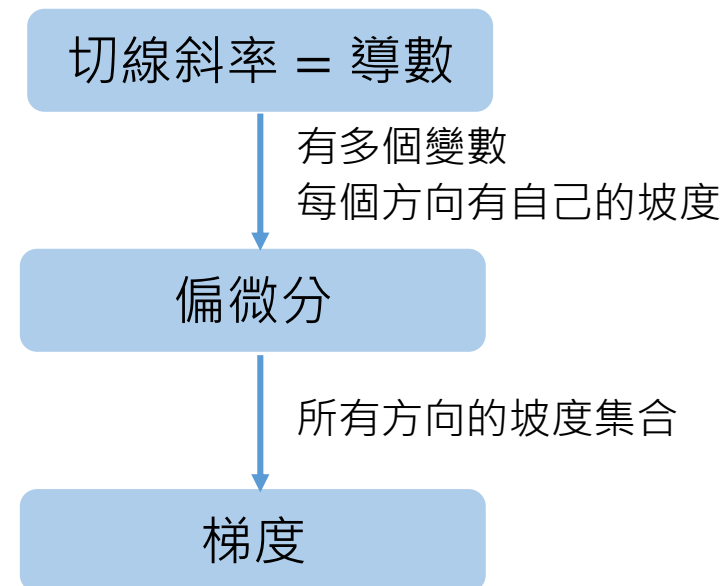
想解決的問題



梯度下降

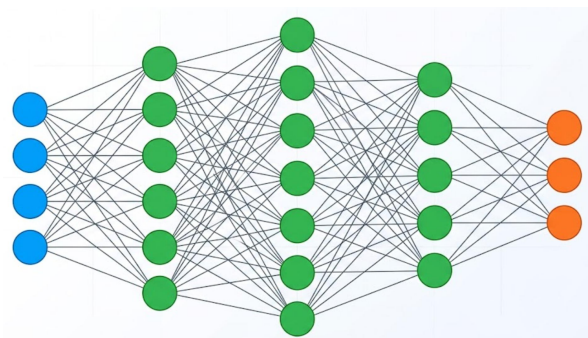
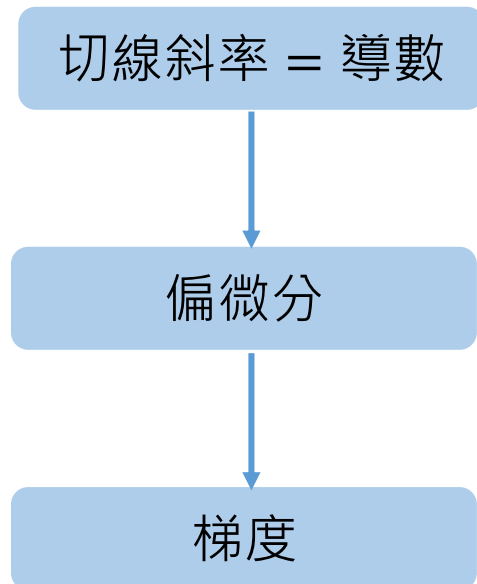


計算坡度

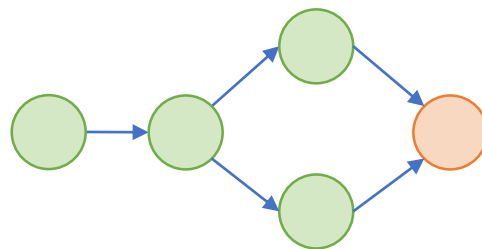
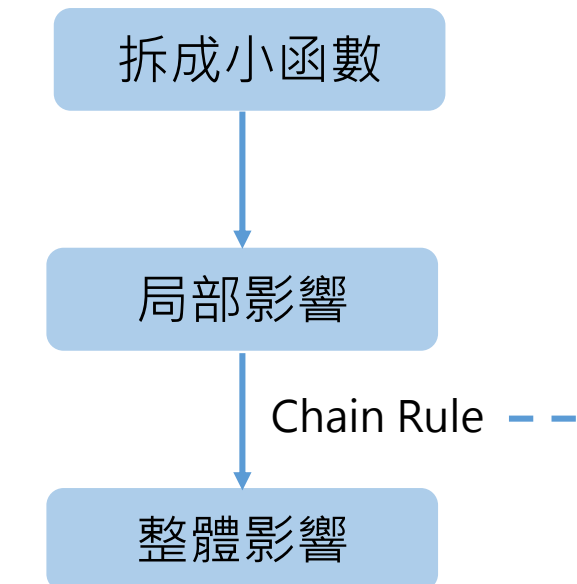


總結（二） - 反向傳播與優化策略

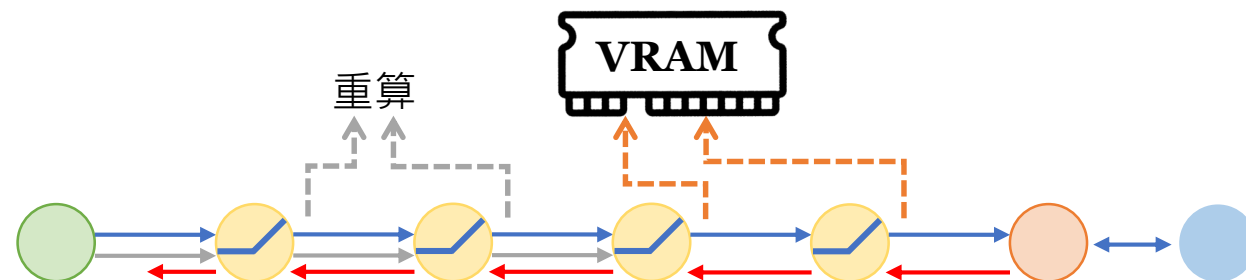
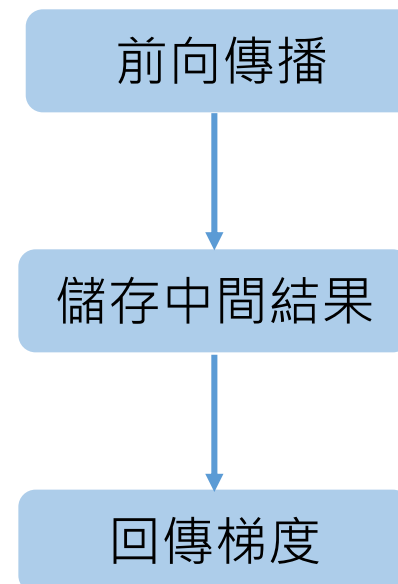
- 計算坡度



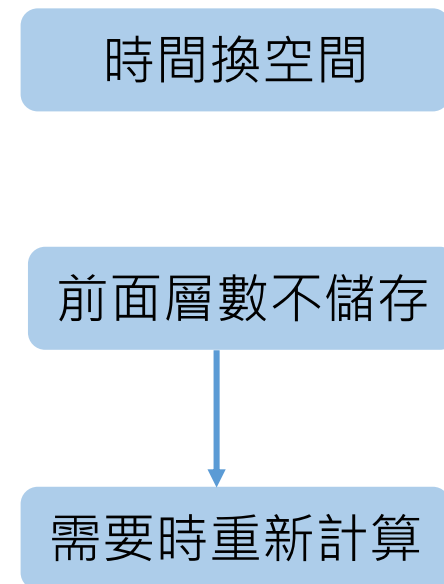
- 複雜函數的梯度



- 反向傳播



- 梯度檢查點

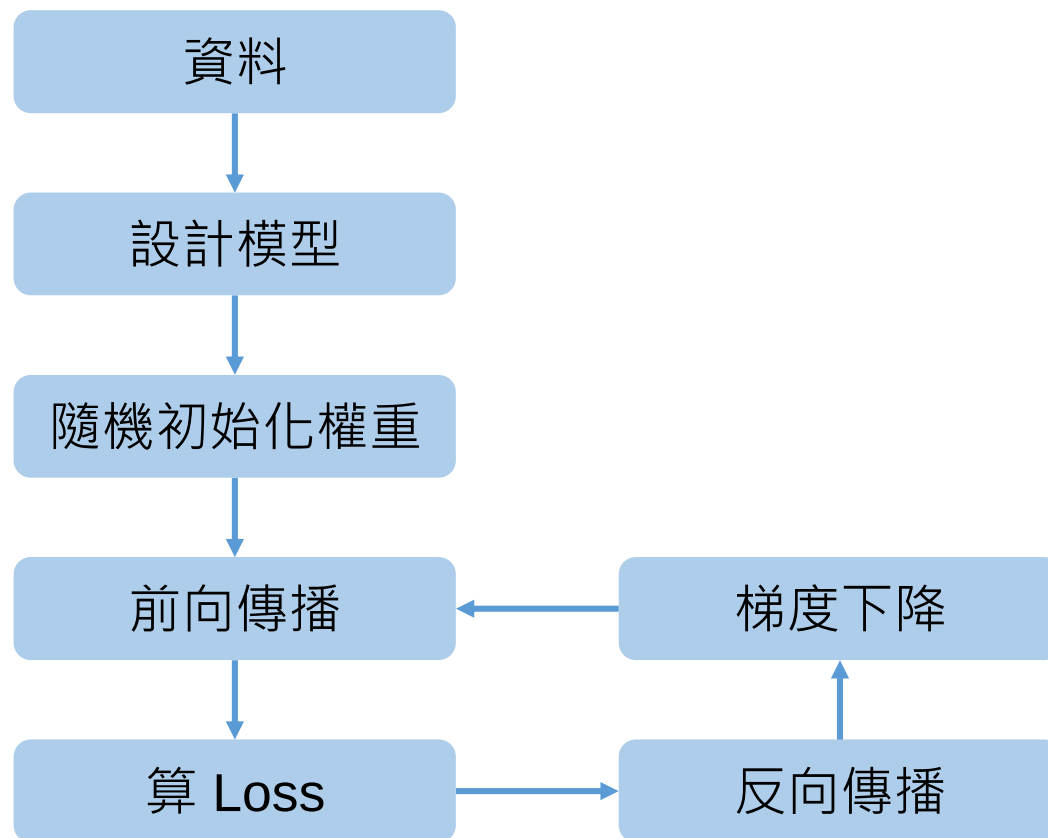


深度學習中的微積分

- 如何有系統地調整參數

知識點	用途
導數	知道坡度
偏微分	知道每個參數的坡度
梯度	知道整體下降方向
Chain Rule	能分析複雜函數
反向傳播	有效率地算 Chain Rule
梯度檢查點	減少反向傳播所需 VRAM

- 模型訓練週期



Chapter 3.

統計

Chapter 3. 統計

Sec. 1 數據輪廓

- 敘述性統計
- 資料判讀

Sec. 2 模型評估

- 迴歸模型
- 分類模型

Sec. 3 資訊理論

- 機率系統
- 不確定性
- 估計分布
- 模型訓練

模型開發流程

資料造就模型



- PM：「新功能讓轉換率提升」
- 數據提供者：「這個特徵很重要」
- 模型訓練良好，上線後表現差

Data are not just numbers,
they are numbers with a context.

David S. Moore

Section 1.

數據輪廓

1. 敘述性統計
2. 資料判讀

平均數與中位數

數據關鍵特徵

- 中心在哪裡
- 分散程度
- 整體分布形狀

情境：新創軟體公司員工薪水

4.5	4.8	5.0	5.0	5.2	5.5	5.7	6.0	6.5	7.0
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- 平均數 = 5.52 、 中位數 = $(5.2 + 5.5)/2 = 5.35$

● CEO 加入

4.5	4.8	5.0	5.0	5.2	5.5	5.7	6.0	6.5	7.0	20
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	----

- 平均數 = 6.8 、 中位數 = 5.5

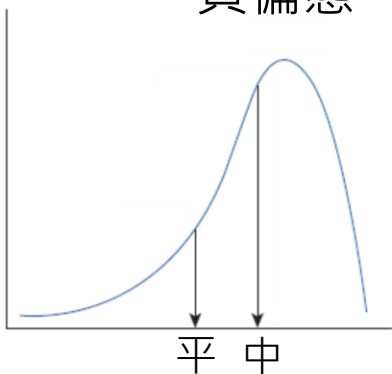
● 平均數 (μ)

- 利用所有資訊
- 適用於對稱資料
- 易受極端值影響

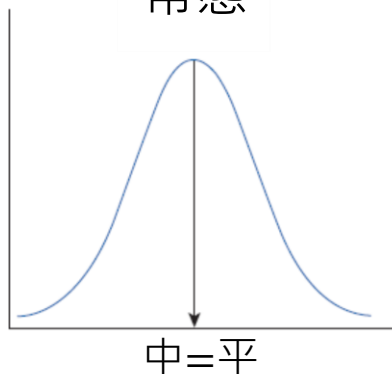
● 中位數

- 選用「典型」數據
- 可用於偏態資料
- 可抵抗極端值

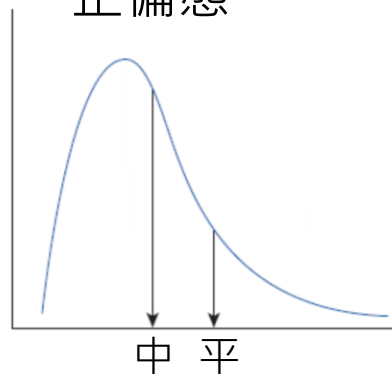
負偏態



常態



正偏態



四分位距 (IQR)

散布情況

- 四分位數：資料排序後切四等分

- $Q1$ = 第 25% 位置的數值
- $Q2$ = 第 50% 位置的數值 (中位數)
- $Q3$ = 第 75% 位置的數值

- IQR (四分位距) = $Q3 - Q1$

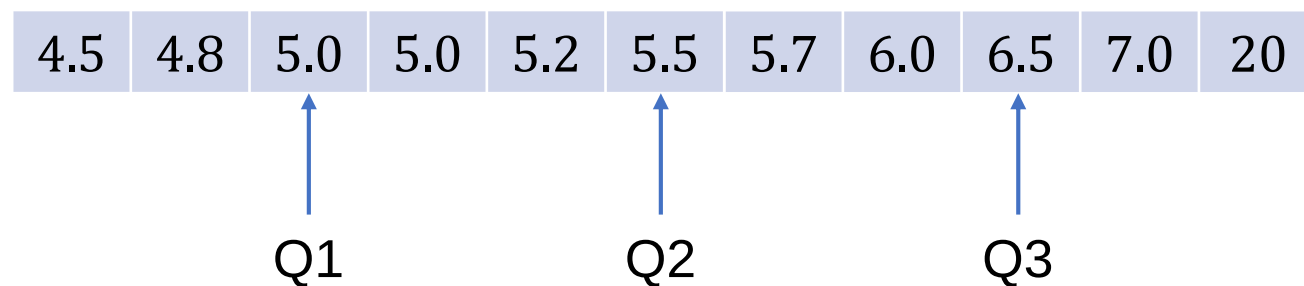
- 中間 50% 的人的分布範圍

- 離群值檢測：

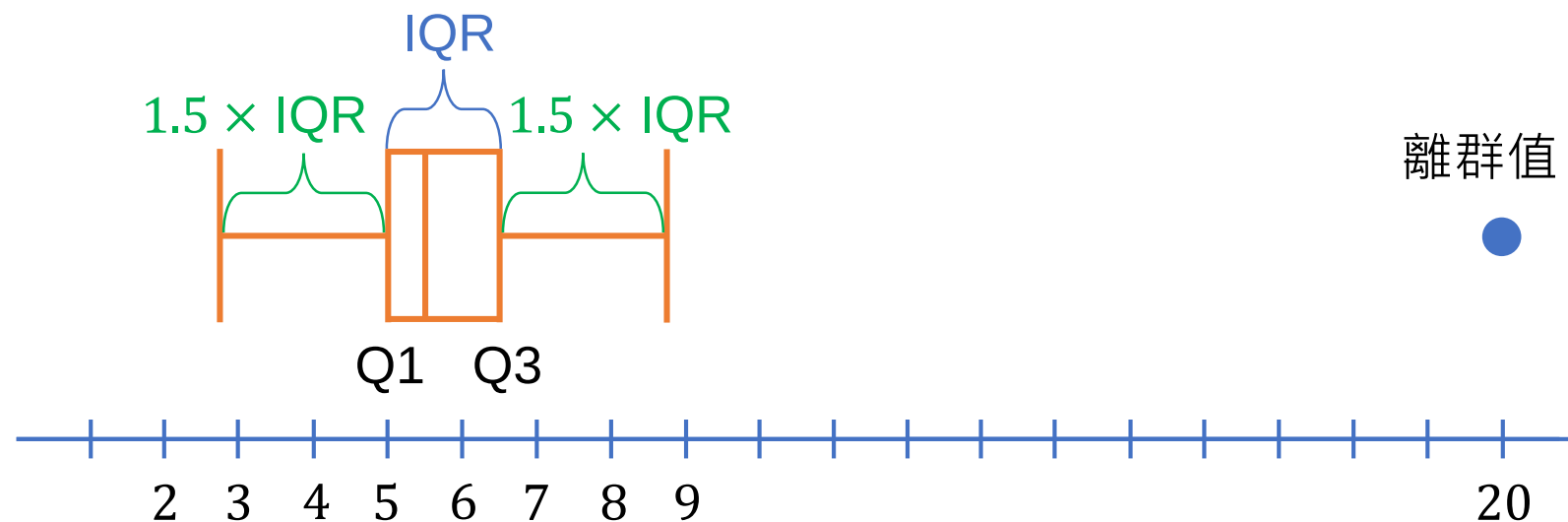
低於 $Q1 - 1.5 \times IQR$ 、

高於 $Q3 + 1.5 \times IQR$

情境：新創軟體公司員工薪水



- 盒鬚圖 (箱形圖 , box plot)



標準差

分散程度：與平均值的距離

- 每個數值與平均數的差距
- 取平方（消除正負差異）
- 取平均
- 開根號（拉回原本單位）

$$\sigma = \sqrt{\frac{\sum (x_i - \text{平均})^2}{N}}$$

(= $\sqrt{\text{變異數}}$)

情境：新創軟體公司員工薪水

4.5	4.8	5.0	5.0	5.2	5.5	5.7	6.0	6.5	7.0	20
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	----

平均 = 6.8

$$\sigma = \sqrt{\frac{(4.5 - 7.75)^2 + (4.8 - 7.75)^2 + \dots + (20 - 7.75)^2}{11}}$$

≈ 4.22

- 移除離群數值

4.5	4.8	5.0	5.0	5.2	5.5	5.7	6.0	6.5	7.0
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

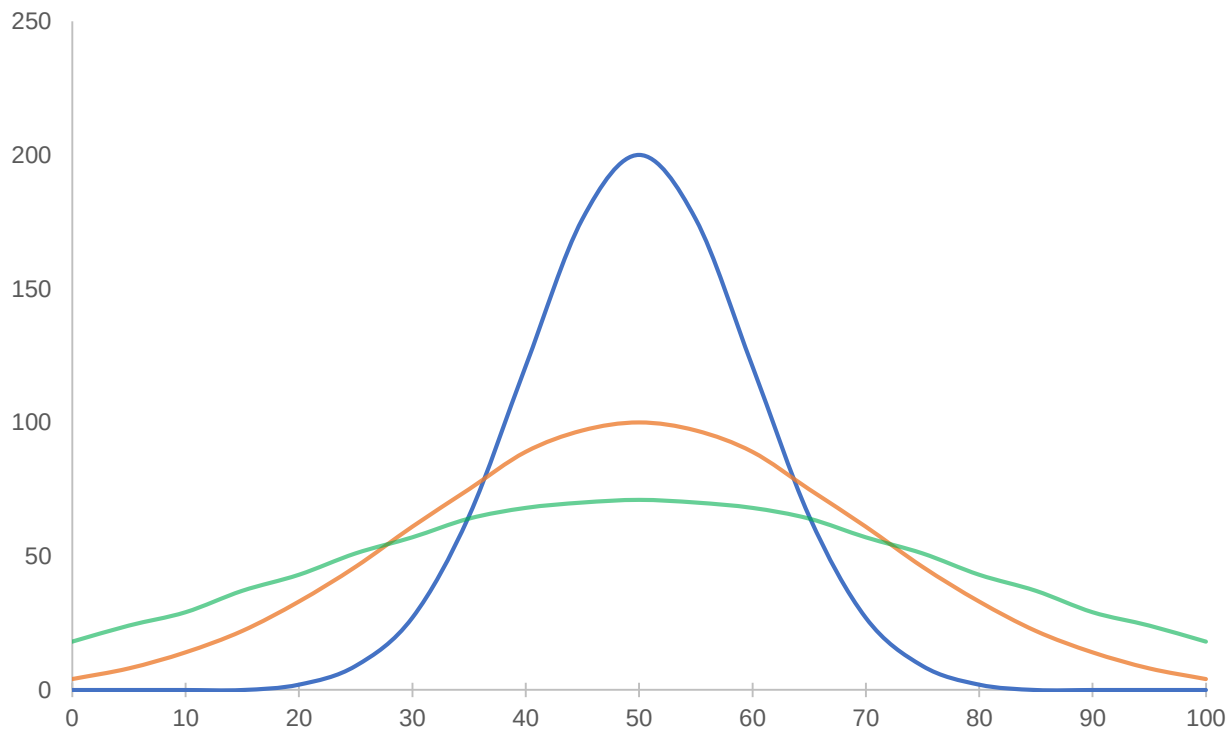
平均 = 5.52

$\sigma \approx 0.75$

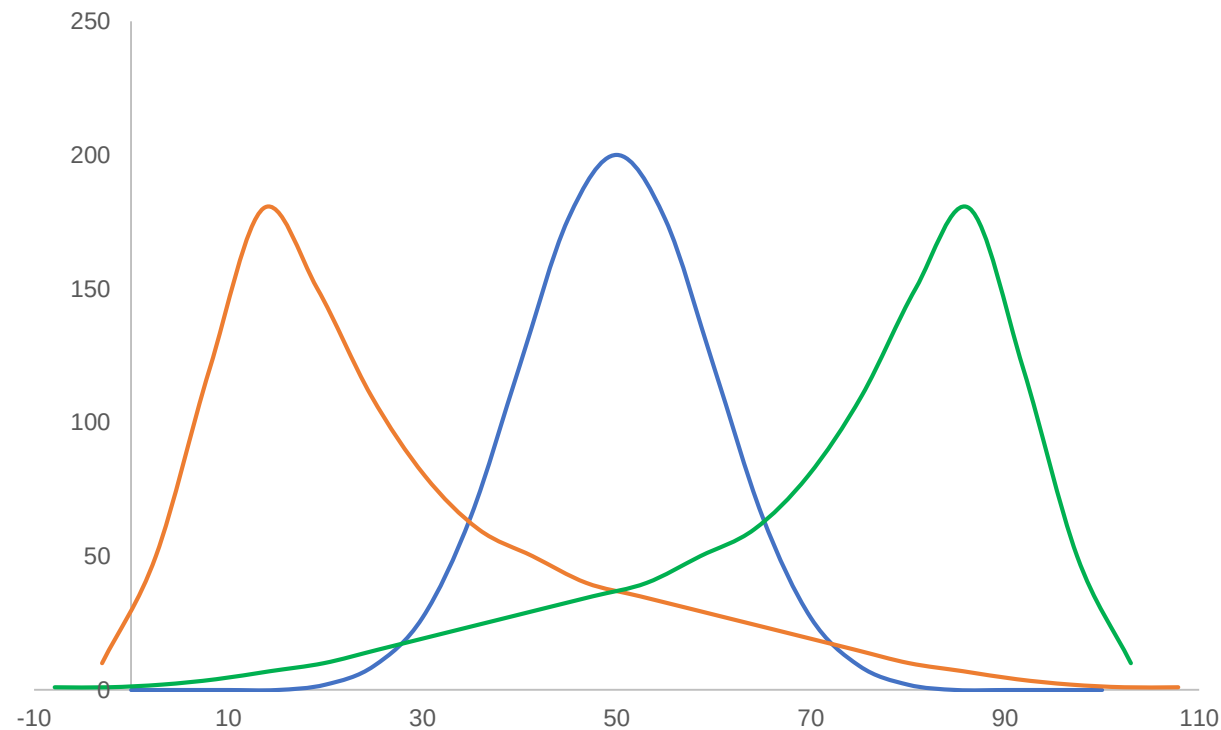
標準差 - 視覺化比較

分散程度：與平均值的距離

- 標準差：10, 20, 30



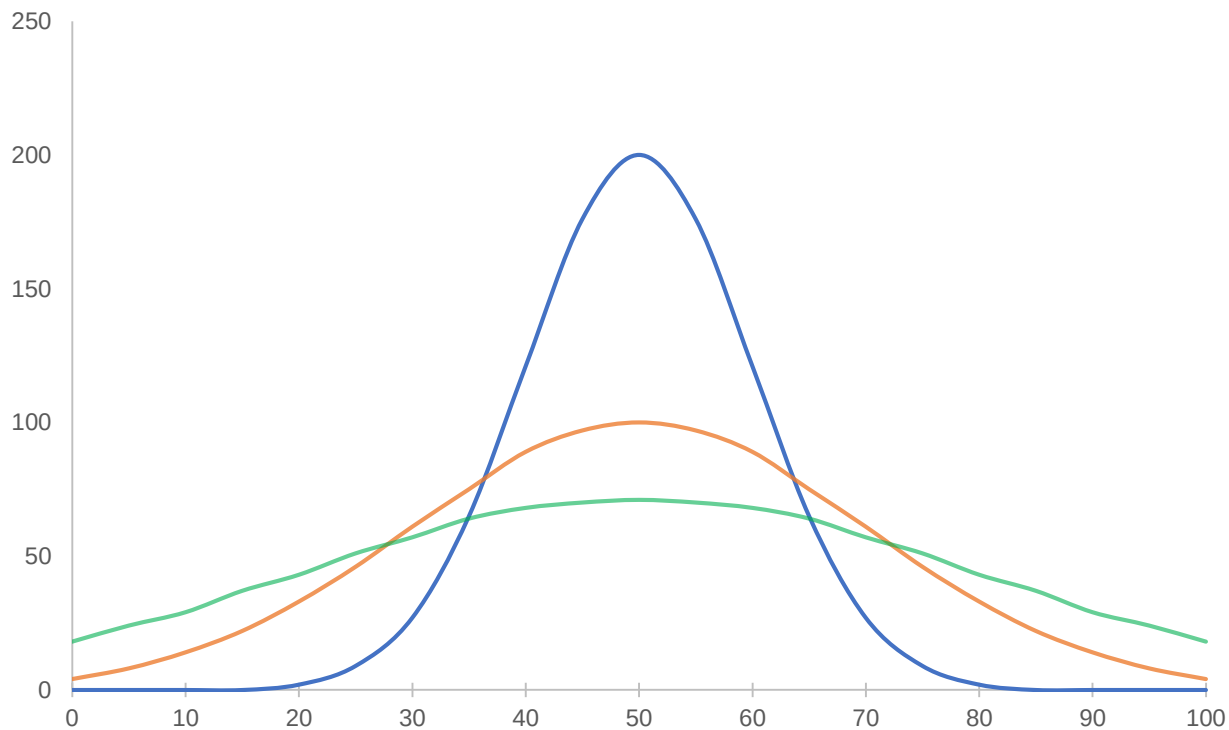
- 標準差：常態分布 10，正偏態 20，負偏態 20



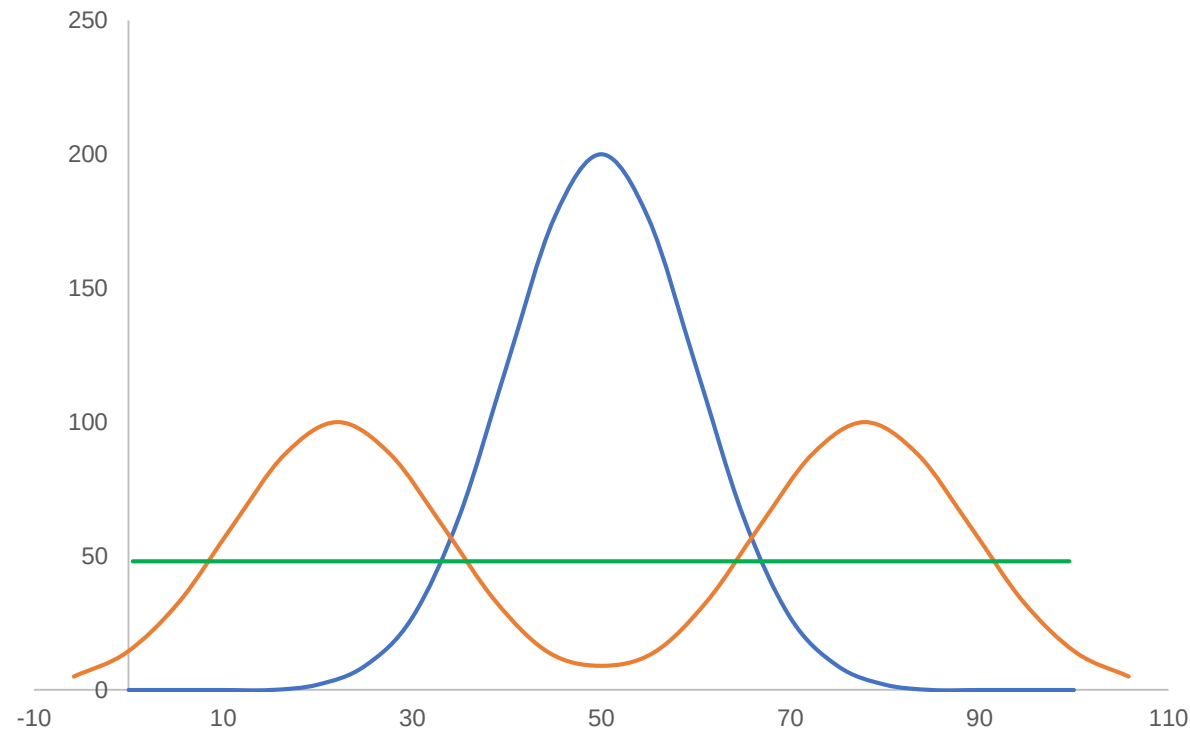
標準差 - 視覺化比較

分散程度：與平均值的距離

- 標準差：10, 20, 30



- 標準差：常態分布 10，M 型 30，均勻 30



統計指標比較

情境：新創軟體公司員工薪水

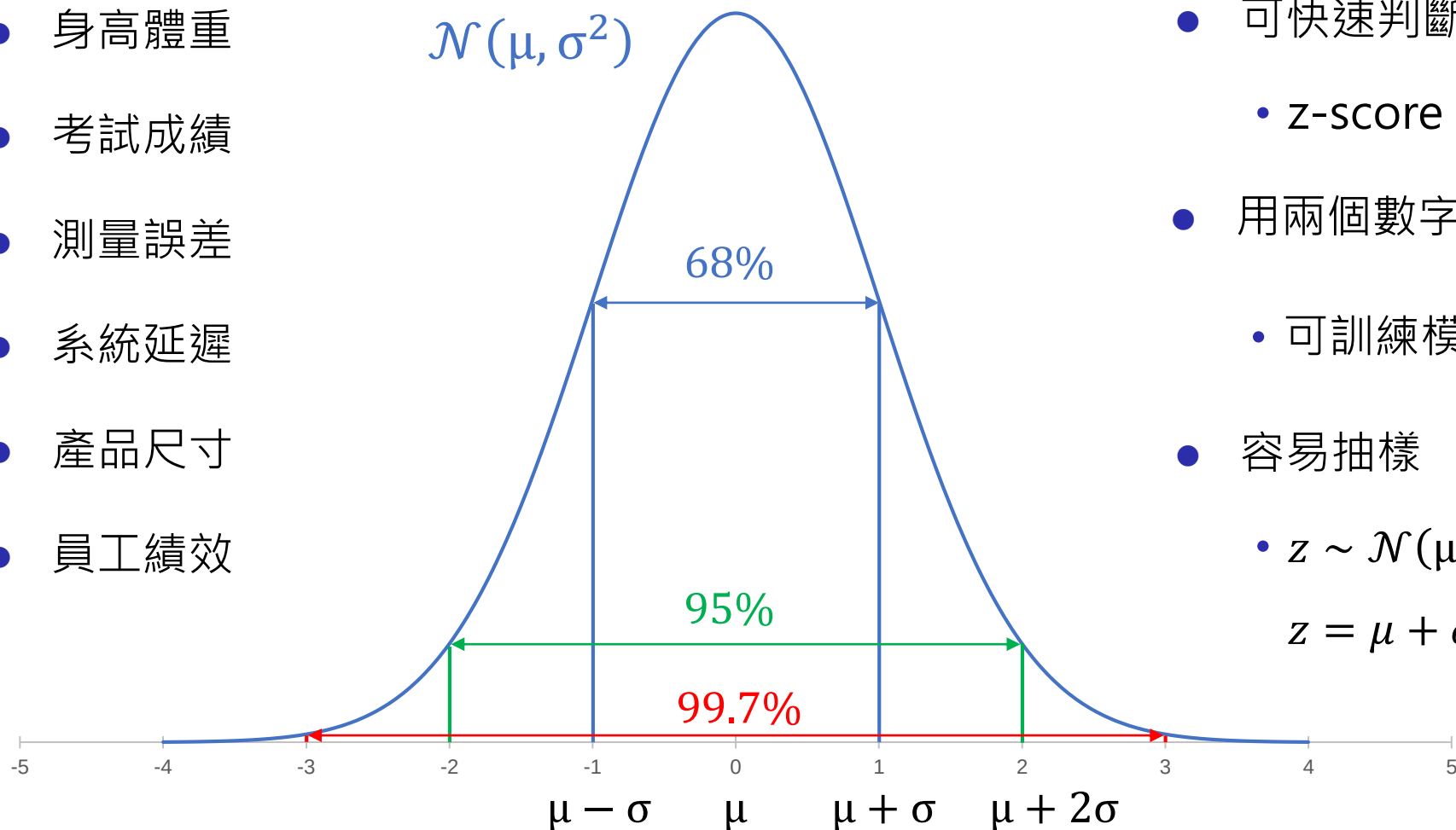
4.5	4.8	5.0	5.0	5.2	5.5	5.7	6.0	6.5	7.0	20
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	----

統計量	原始	排除離群值	變化幅度
平均數	6.8	5.52	−18.8%
中位數	5.5	5.35	−2.7%
IQR	1.5	1.35	−10%
標準差	4.22	0.75	−82.2%

常態分布 / 高斯分布 (Gaussian)

自然現象的分布

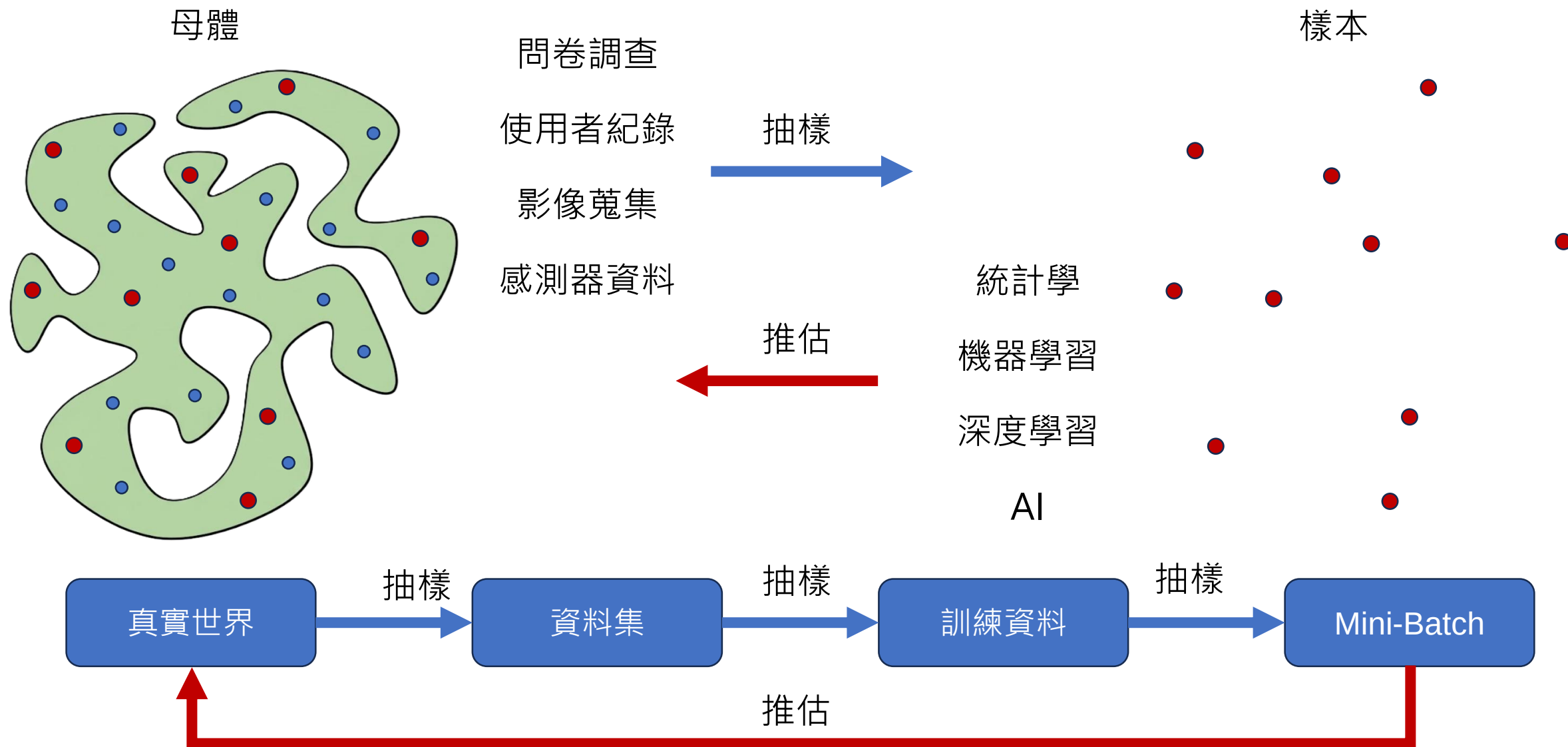
- 身高體重
- 考試成績
- 測量誤差
- 系統延遲
- 產品尺寸
- 員工績效



特性

- 可快速判斷離群值
 - z-score：距離平均值幾個標準差
- 用兩個數字描述唯一的分布
 - 可訓練模型輸出 $\begin{bmatrix} \mu \\ \sigma \end{bmatrix}$ ，擬合目標分布
- 容易抽樣 `np.random.normal()`
 - $z \sim \mathcal{N}(\mu, \sigma^2)$ 等價於
 $z = \mu + \sigma \cdot \epsilon$ ，其中 $\epsilon \sim \mathcal{N}(0, 1)$

母體與抽樣



偏誤 (Bias) - 選擇偏誤

選擇偏誤：資料蒐集的方式，讓某些類型被過度或過少代表

- 自我選擇偏誤：只蒐集到「願意參加調查」者的意見，非完全隨機抽選

- 商品、店家評價
- 問卷調查

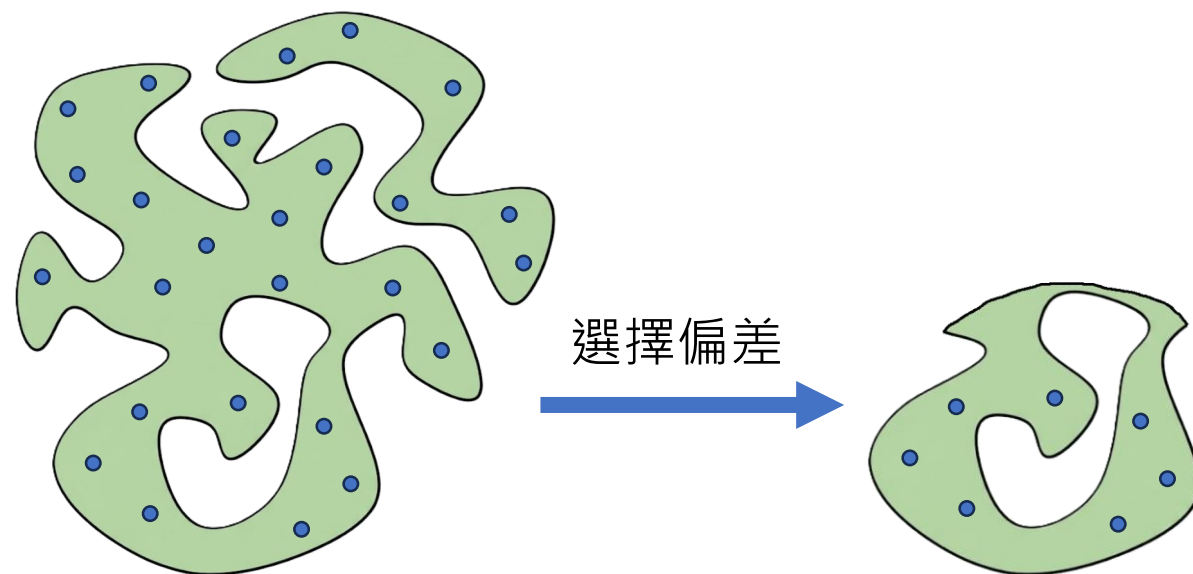
只蒐集到意見極端的人

忽視懶得寫評價、問卷的多數人

- 倖存者偏誤：

- 對二戰成功返航的戰機，分析中彈部位
- 比爾·蓋茲、賈伯斯輟學創業成功
- 減肥 / 養生偏方成功案例

死人不會說話



偏誤 (Bias) - 分布偏移

瞬息萬變的世界

選擇偏誤：資料抽錯

分布偏移：世界變了

- 特徵偏移

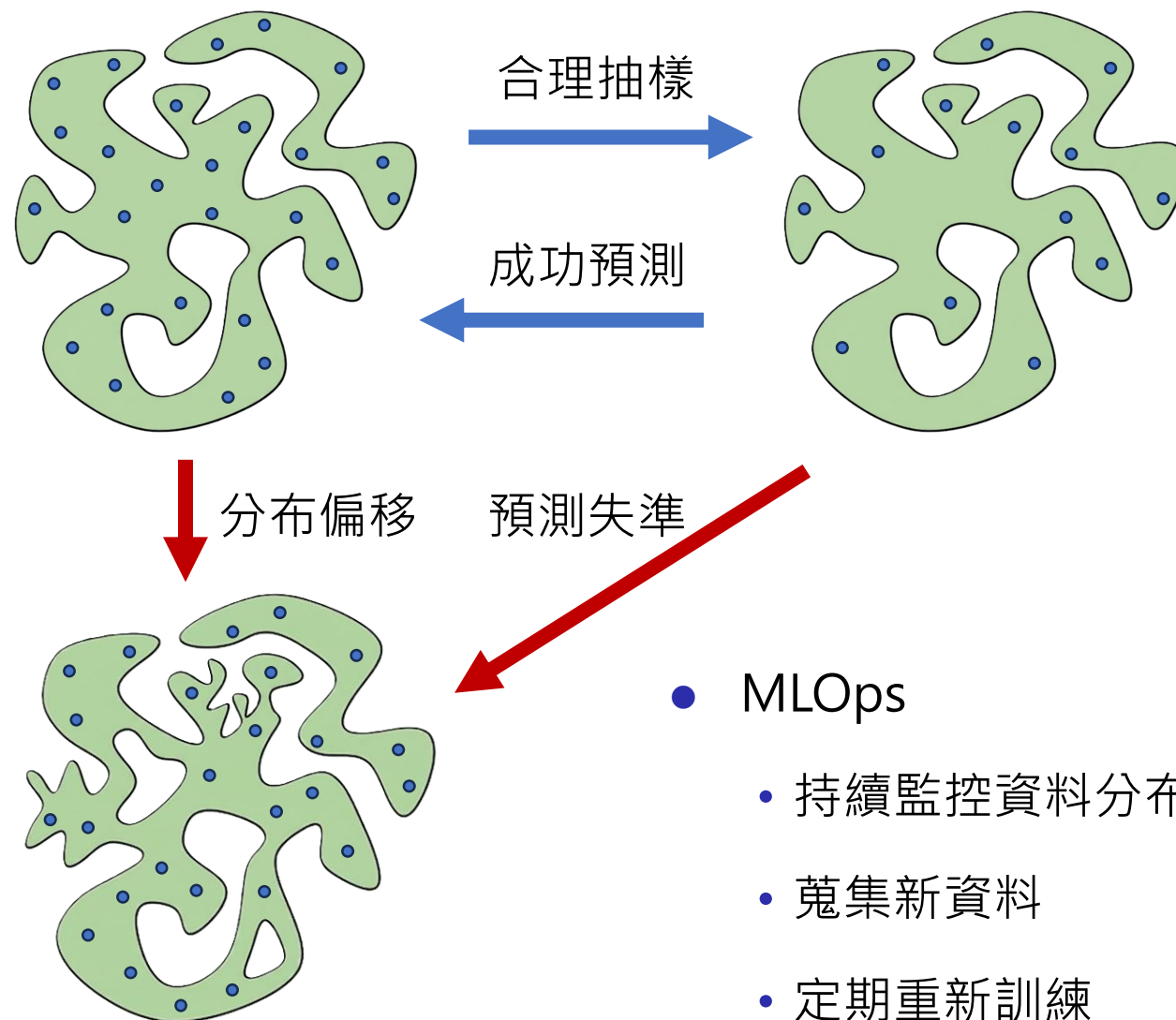
- 使用乾淨影像訓練自動駕駛，上路後遇到大雨、大雪、無路燈、鏡頭髒汙

- 標籤偏移

- 疫情期間發燒者高機率確診，當時訓練的模型放到現在容易誤診

- 概念偏移

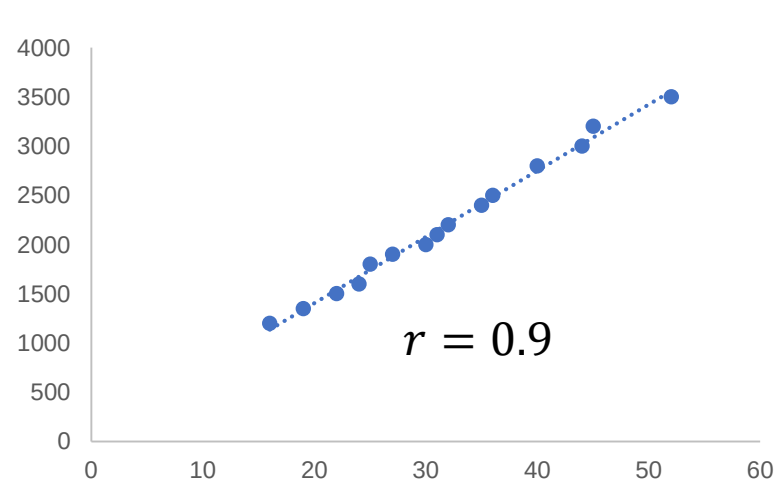
- 詐騙信件偵測，詐騙集團發現成功率降低後改變詐騙方式



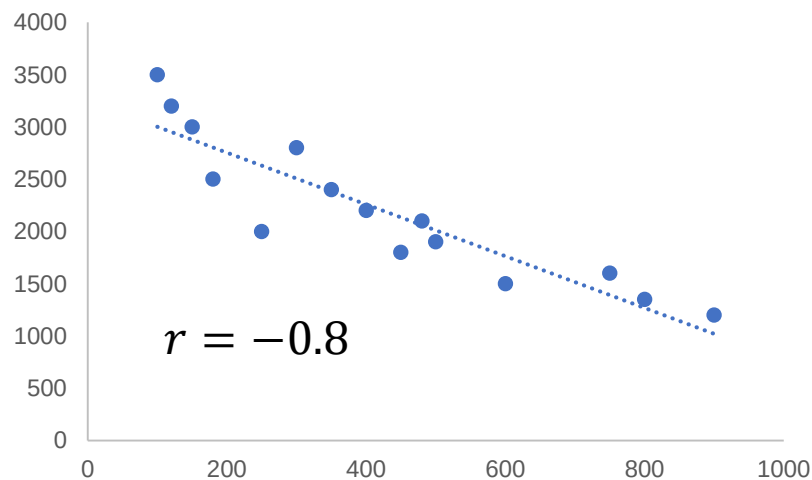
相關係數

變數之間的線性關聯強度

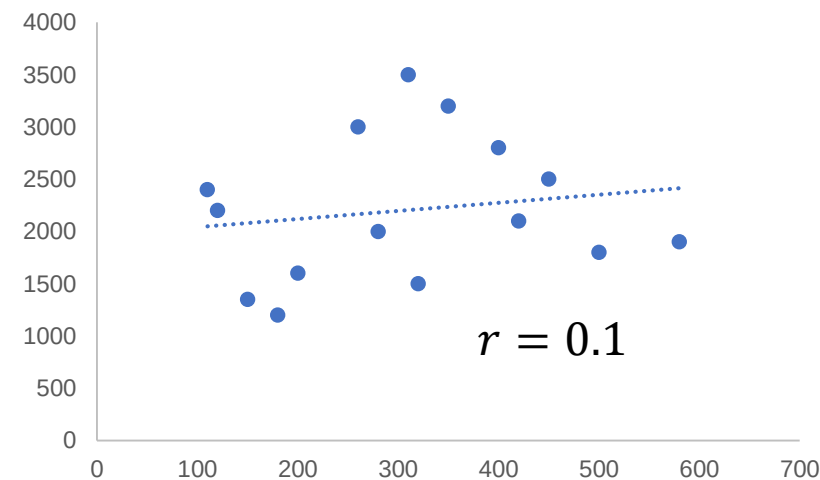
- 皮爾森相關係數 (r)：衡量兩連續變數之間「線性關聯」的強度與方向
 - 正相關： r 接近 1，兩者等比例同向變化
 - 負相關： r 接近 -1 ，兩者等比例反向變化
 - 無相關： r 接近 0，兩者變化無比例關係



正相關：房價與坪數



負相關：房價與捷運站距離



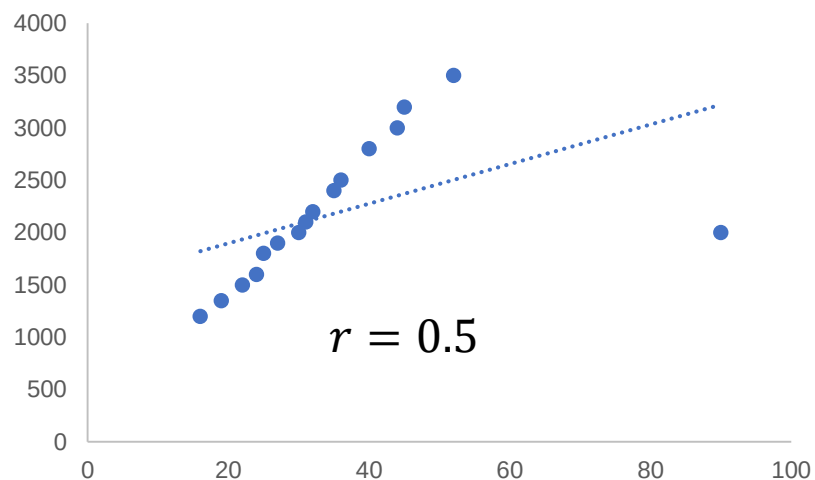
無相關：房價與便利商店距離

相關係數 - 注意事項

變數之間的線性關聯強度

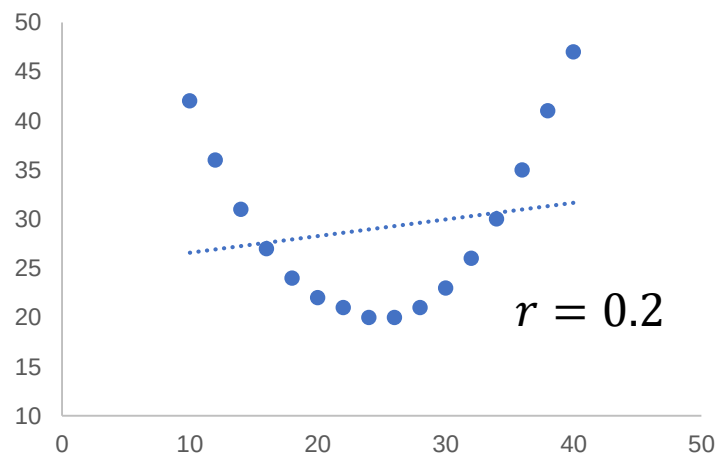
- 皮爾森相關係數 (r) 注意事項

- 容易被極端值影響



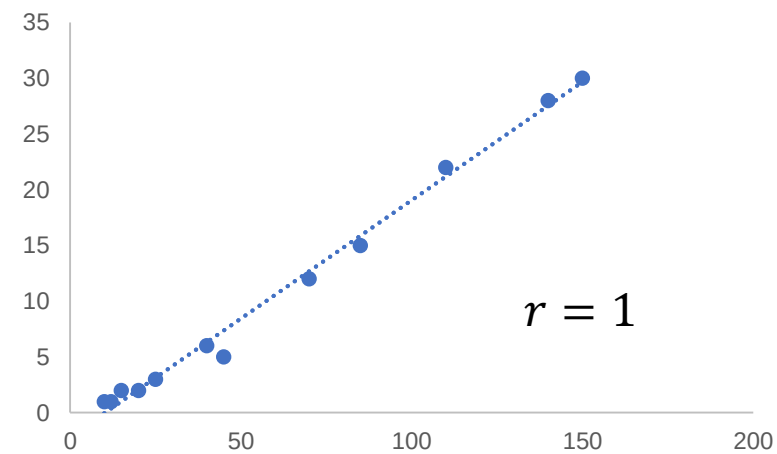
一筆極端值使 r 從 0.9 變 0.5

- 只能捕捉線性關係



U 型分布：用電量與氣溫

- 相關不等於因果



- 溺水人數與冰淇淋銷量高度相關
- 巧克力消費量 vs 諾貝爾獎得主

p-value - 概念介紹

統計指標是否也來自運氣？

- 案例：廣告費用與業績相關係數 = 0.4，代表兩者真的有關係嗎？

- A：有關，抽樣如實反映真實情況
- B：無關，只是剛好抽到看似有關的樣本。其他樣本相關係數可能很低

無法排除這種可能性，但可量化可能性大小

- p-value：剛好抽到這種樣本的機率是多少

- p-value 小，代表「不太可能只是剛好抽到」、「兩者無關這個說法有問題」
⇒ 傾向「拒絕虛無假設」，認為應該「真的有相關 / 有差異 / 有效果」

p-value：無罪推定，再檢視證據是否為巧合、是否足以推翻無罪假設

相關係數：關係的強弱

P-value：此關係是否可信

虛無假設：假設

「沒有相關」、

「沒有差異」、

「沒有效果」

p-value - 範例一

統計指標是否也來自運氣？

- 案例：電商網站按鈕顏色測試

- 舊按鈕（藍色）長期轉換率：5%

- 新按鈕（橘色）測試轉換率：200 位訪客測試，其中 16 位下單 $\Rightarrow$ 轉換率 8%

$\Rightarrow p\text{-value} = 0.02$

$\Rightarrow$ 假設「按鈕顏色不影響轉換率」，那在 200 人的樣本中，恰好觀察到 8% 以上轉換率的機率是 2%

慣例上，p-value 門檻設為 0.05 (5%)

$\Rightarrow$ 機率太低，假設可能不成立（拒絕虛無假設）

$\Rightarrow$ 傾向於相信「橘色按鈕真的比較好」（效果 / 差異顯著）

p-value - 範例二

統計指標是否也來自運氣？

- 案例：電商網站按鈕顏色測試

案例替換：新舊按鈕 → 新舊推薦系統

- 舊按鈕（藍色）長期轉換率：5%
- 新按鈕（橘色）測試轉換率：200 位訪客測試，其中 12 位下單 $\Rightarrow$ 轉換率 6%

$\Rightarrow p\text{-value} = 0.26$

$\Rightarrow$ 假設「按鈕顏色不影響轉換率」，那在 200 人的樣本中，恰好觀察到 6% 以上轉換率的機率是 26%

$\Rightarrow$ 機率不低，不足以推翻「顏色不影響」的假設（無法拒絕虛無假設）

無法拒絕 $\neq$ 接受

✗ 橘色按鈕沒用

✓ 不足以證明有用（效果 / 差異**不顯著**）

p-value - 常見誤區

統計指標是否也來自運氣？

-  p-value = 虛無假設成立的機率
(「新按鈕沒用」成立的機率)
-  p-value = 若虛無假設成立，看到此現象的機率
(若新按鈕沒用，看到轉換率提升的機率)
-  p-value 愈小，效果愈大
-  p-value 只反映「這個現象應該不是巧合」，不反映效果大小
(若樣本數夠大，會讓微小差異也變顯著)
-  p-value 大 (不顯著) = 證明沒效果
-  只是證據不足 (無法拒絕虛無假設 $\neq$ 接受)

p-value - 應用場景

統計推論：確認世界的運作

	統計推論	機器學習
核心關注	世界是不是這樣運作	能不能準確預測
目標	解釋性與因果	泛化能力
面對未知	量化不確定性	接受黑箱模型
應用場景	A/B Testing、特徵篩選	推薦系統、影像辨識、LLM
實務問題	「廣告是否真的提升轉換率？」	「哪位使用者最可能購買？」

數據輪廓總結

資料判讀流程

- 快速了解數據中心點：平均數、中位數
- 數據分散程度和離群值判斷：IQR、標準差
- 確認資料代表性：取得的資料分布與母體 / 目標是否有偏差
- 判斷變數線性相關性：相關係數
- 質疑判斷結果：相關性背後是否有其他因素？是否只是抽樣造成的差異？

Section 2.

模型評估

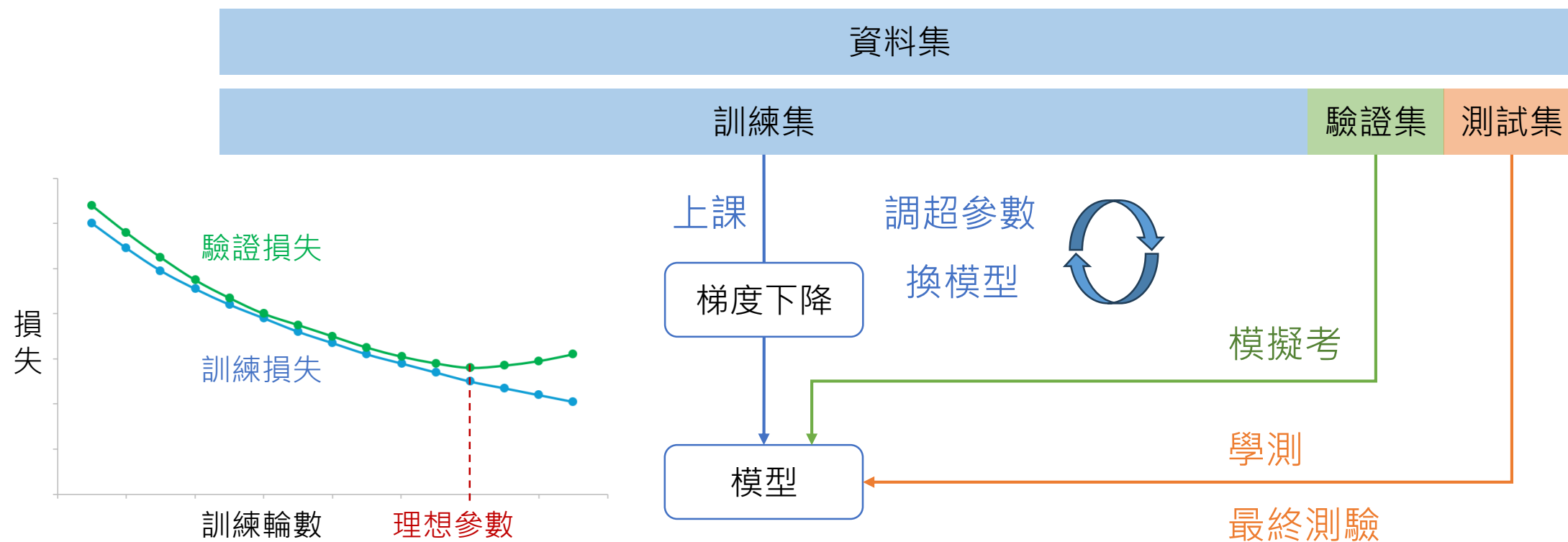
1. 評估原則
2. 迴歸模型
3. 分類模型

模型評估

什麼樣的模型才算好？



- 梯度下降可得到損失較低的參數組合（在訓練資料上的損失）
- 真正關心：面對**沒看過的資料**，能不能做好預測



R^2 - 迴歸模型評估

模型如何表現才叫做準？

- 迴歸模型預測連續數值 (房價、票房、營收、降雨機率、良率、馬拉松完賽時間)
- 問題：預測誤差 50 萬元，算準嗎？ $\Rightarrow$ 需有比較基準
- 「瞎猜平均值」當基準：把資料取平均，當作每一筆新資料的預測結果
- R^2 ：模型比「瞎猜平均值」好多少？

$$R^2 = 1 - \frac{\text{模型預測值的誤差}}{\text{瞎猜平均值的誤差}} = 1 - \frac{\sum (y_i - \text{預測值})^2}{\sum (y_i - \text{平均值})^2} = 1 - \frac{\text{MSE}}{\text{變異數}}$$

$$= 1 - \frac{\text{有模型後還剩多少混亂程度}}{\text{資料本身的分散 / 混亂程度}}$$

模型的解釋力

$R^2 = 1$ ：模型解釋了所有的變異

$R^2 = 0$ ：模型跟瞎猜平均值一樣

$R^2 < 0$ ：模型比瞎猜平均值還差

R^2 - 案例說明

案例：房價預測

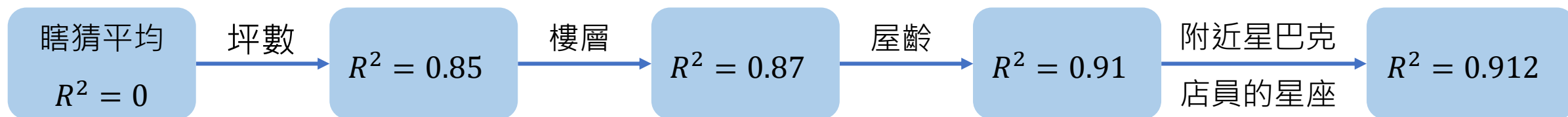
- 沒有模型時：只用平均房價猜，誤差平方和 = 500 萬
- 根據坪數訓練線性模型，誤差平方和 = 75 萬
- $R^2 = 1 - \frac{75}{500} = 0.85$

模型解釋了 85% 的房價差異，代表坪數確實與房價相關，考慮坪數後，房價更容易預測了。
但還有 15% 無法解釋，可能是地段、樓層、屋齡等因素造成

相關不等於因果！

R^2 - 盲點

不反對任何變數



- 變數增加不會使 R^2 減低

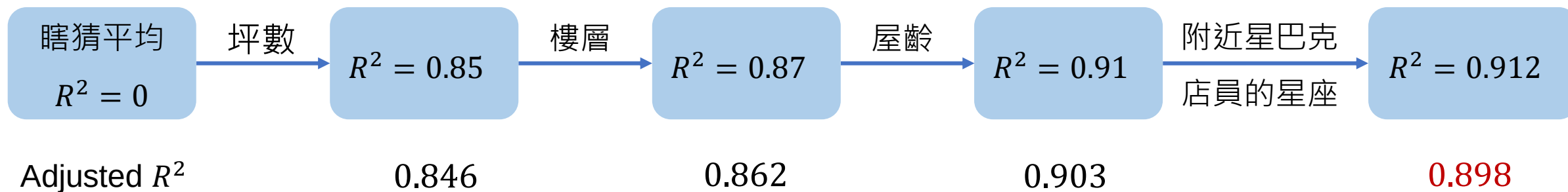
- 若模型認為新變數與預測項（如房價）無關，可選擇無視此變數， R^2 不變
- 若新變數正好與預測項有一點點相關性（可能只是樣本巧合），會使 R^2 稍微增加

此「樣本巧合造成的相關性」不適用於一般情況（無法泛化），會誤導模型，對新資料預測失準
過擬合（Overfitting）

Adjusted R^2

變數必須要有貢獻

- Adjusted R^2 ：每增加一個變數，都必須有「相應的貢獻度（解釋力）」才加分，否則倒扣
 - 若模型認為新變數與預測項（如房價）無關，可選擇無視此變數， R^2 不變



R^2 ：有沒有進步
Adjusted R^2 ：進步值不值得

混淆矩陣

模型如何表現才叫做準？

- 分類模型預測類別（垃圾信件偵測、詐欺偵測、疾病篩檢、影像分類、聲音分類）
- 問題：高準確率（ $\text{Accuracy} = \text{預測正確} / \text{總數}$ ），就夠了嗎？

- 情境：1000 人做快篩，20 人真正確診

模型無腦猜「沒病」 $\Rightarrow$ 準確率 = $980/1000 = 98\%$ $\Rightarrow$ 需要更精細的指標

- 混淆矩陣
(Confusion Matrix)

	預測 Positive	預測 Negative
實際 Positive	TP：真陽性 (True Positive) 成功抓出	FN：假陰性 (False Negative) 漏網之魚
實際 Negative	FP：假陽性 (False Positive) 虛驚一場	TN：真陰性 (True Negative) 正確排除

Precision 與 Recall

- 情境：1000 人做快篩，20 人真正確診

	預測 Positive	預測 Negative
實際 Positive	TP = 15 成功抓出	FN = 5 漏網之魚
實際 Negative	FP = 20 虛驚一場	TN = 960 正確排除

- 準確率 = $(15 + 960)/1000 = 97.5\%$
- 準確率低於「無腦」模型，但顯然較有用

- Precision (精確率)：模型講的話是否可信
 - 直覺定義：模型說陽性時，有多少真的陽性
- Recall (召回率)：模型是否抓得夠全
 - 直覺定義：真的陽性的人，有找出多少

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{15}{15 + 20} = 42.9\%$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{15}{15 + 5} = 75\%$$

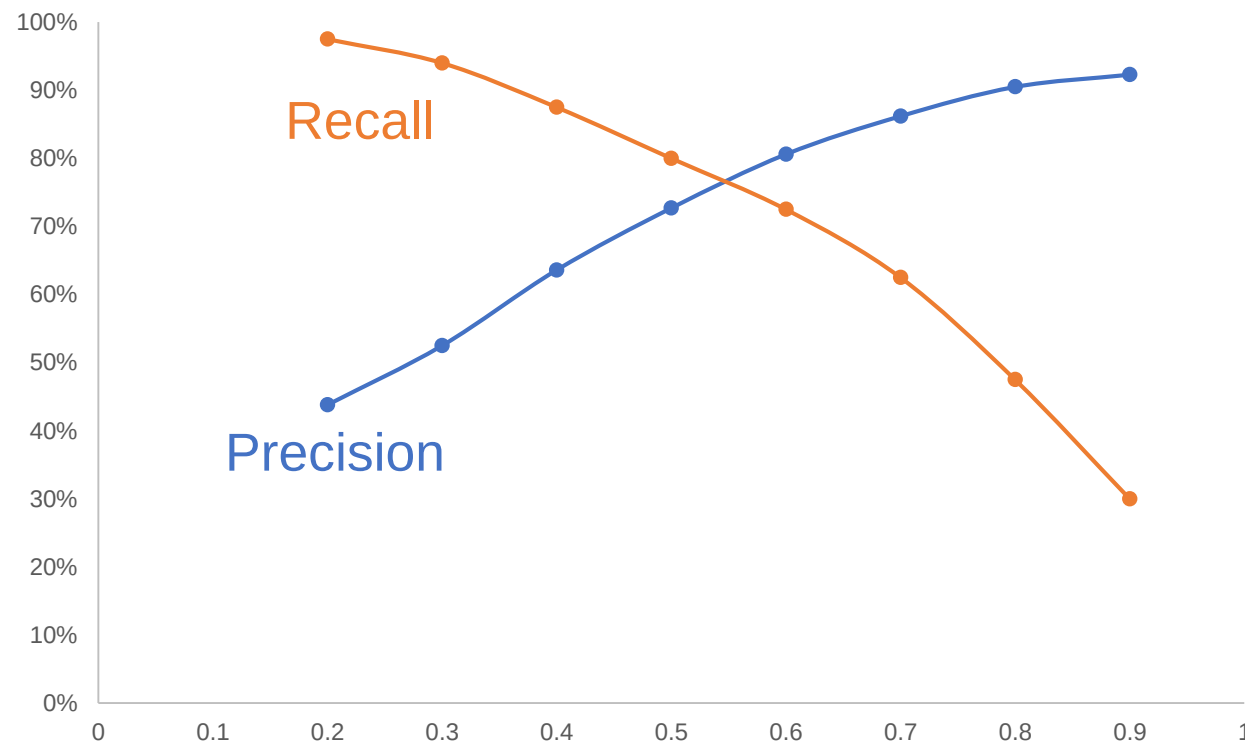
Precision 與 Recall 的權衡

- 分類模型實際輸出不是 Yes / No，而是機率



- Threshold 由人決定 (視應用場景決定)

- Threshold 低：稍有嫌疑就抓出來 $\Rightarrow$ 高 Recall、低 Precision $\Rightarrow$ 寧可錯殺，絕不放過
- Threshold 高：嫌疑很大才抓出來 $\Rightarrow$ 高 Precision、低 Recall $\Rightarrow$ 寧缺勿濫，絕對把握



Precision 與 Recall 的權衡

- Threshold 由人決定 (視應用場景決定)

- Threshold 低：稍有嫌疑就抓出來

⇒ 高 Recall、低 Precision

⇒ 寧可錯殺，絕不放過

- Threshold 高：嫌疑很大才抓出來

⇒ 高 Precision、低 Recall

⇒ 寧缺勿濫，絕對把握

高 or 低？

癌症篩檢

詐欺偵測

推薦系統

人臉辨識

火災警報

垃圾信件

設備故障預測

內容安全審查

F1-score

只想要一個綜合分數

案例：Precision = 42.9% , Recall = 75%

- 算術平均：(42.9% + 75%) / 2 $\approx$ 59%

- F1-score：調和平均 = 倒數的平均再取倒數 $= \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2P \cdot R}{P + R} \left(= \frac{2TP}{2TP + FP + FN} \right)$
 $\approx$ 54.6%

- 特性：偏向較小的值 （兩邊都要及格，若有一者偏低，會放大懲罰）

案例：Precision = 95% , Recall = 5% $\Rightarrow$ 算術平均 = 50%

F1-score = 9.5% （較真實的分數）

模型評估總結

- 模型評估原則：在模型沒看過的數據上，表現如何
- 迴歸模型：預測數值
 - R^2 ：模型的解釋力（解釋了多少資料之間的差異）
 - Adjusted R^2 ：篩選變數，考量每個變數對解釋力的貢獻
- 分類模型：預測類別
 - Accuracy：資料平衡時堪用，不平衡時會失真
 - 混淆矩陣：拆解四種結果 $\Rightarrow$ 根據業務場景 / 需求，決定要關注的指標
 - Recall 高：寧可錯殺 $\Rightarrow$ F1：平衡兩者
 - Precision 高：寧缺勿濫

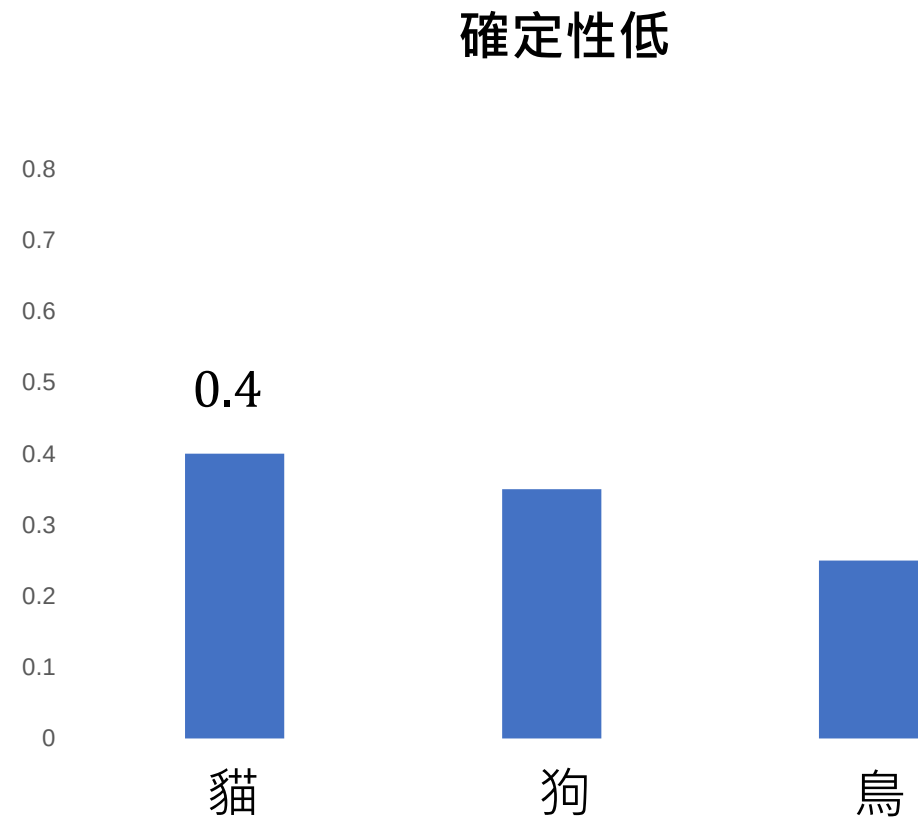
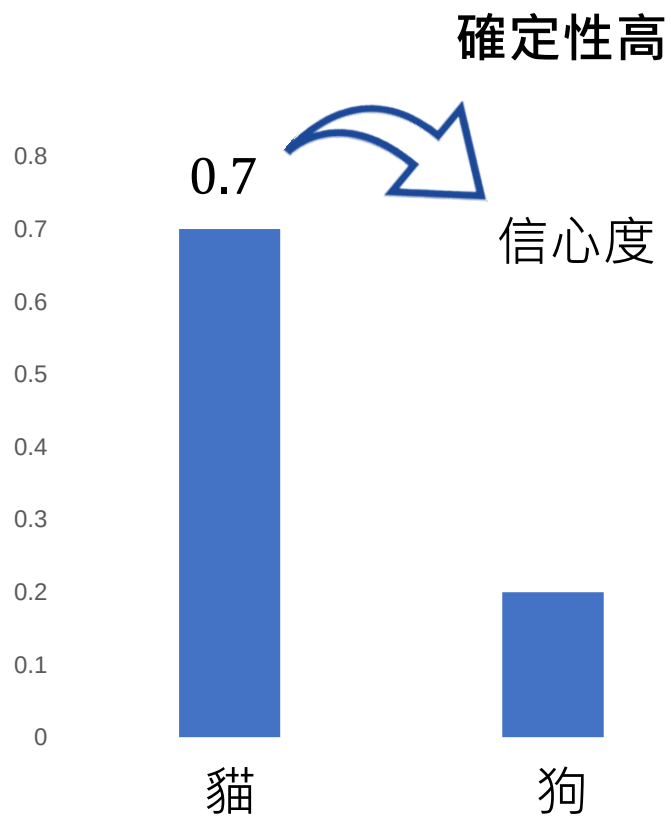
Section 3.

資訊理論

1. 資訊量
2. Entropy
3. Cross Entropy
4. KL-Divergence
5. 語言模型訓練

機率分布

- 把總共 100% 的可能性，分配給所有可能結果
- 影像分類模型：吃一張圖片，輸出每個類別的機率（輸出向量，再過 Softmax）



資訊量、意外程度、機率

- 情境一：氣象預報

- 「台北明天 (7/30) 會很熱」 $\Rightarrow$ 毫無資訊量 (毫不意外) (發生機率極高)
- 「台北明天 (7/30) 會下雪」 $\Rightarrow$ 資訊量很大 (非常意外) (發生機率極低)

- 情境二：擲硬幣

- 一次正面 $\Rightarrow$ 資訊量小 (不太意外) (機率 50%)
- 十次正面 $\Rightarrow$ 資訊量大 (非常意外) (機率 $1/1024$)

「資訊量大小」與「發生機率」是反向關係

系統不確定性

- 兩個袋子各裝 100 顆球

- 袋子 A：100 顆全是紅球

⇒ $p(\text{紅}) = \text{抽到紅球的機率} = 100\%$

⇒ 結果很好猜，沒有不確定性

- 袋子 B：50 顆紅、50 顆藍

⇒ $p(\text{紅}) = p(\text{藍}) = 50\%$

⇒ 只能瞎猜，不確定性高

整體不確定性由「單一事件資訊量」組合而成

量化不確定性

尋找函數 f : 機率 $\rightarrow$ 資訊量

- 線索 1 : 機率低 $\Rightarrow$ 資訊量大

$\Rightarrow$ 函數 $f(p)$ 應是遞減函數

- 線索 2 : 資訊量應可加

(硬幣投出正面，同時骰子擲到 6 點，兩者資訊量應相加)

- 從機率觀點，同時發生是兩者機率相乘

$\Rightarrow$ 函數 f 應把相乘變相加

$$f(p_1 * p_2) = f(p_1) + f(p_2) \Rightarrow f \text{ 應包含 } \log$$

$$(\log(a * b) = \log a + \log b)$$

$$(2^3 = 8, \log_2 8 = 3)$$

- 資訊量 $I(p) = -\log_2(p)$

- 為了遞減，且讓資訊量是正數，所以取負號
- 為了對應電腦運算，所以選擇 $\log$ 以 2 為底

量化不確定性

資訊量 $I(p) = -\log_2(p)$

- 兩個袋子各裝 100 顆球

- 袋子 A : 100 顆全是紅球

$\Rightarrow p(\text{紅}) = \text{抽到紅球的機率} = 100\%$

$\Rightarrow \text{抽到紅球的資訊量}$

$$I(p(\text{紅})) = -\log_2(100\%) = -\textcolor{red}{0}$$

$$(\textcolor{red}{2}^{\textcolor{red}{0}} = 1)$$

- 袋子 B : 50 顆紅、50 顆藍

$\Rightarrow p(\text{紅}) = p(\text{藍}) = 50\%$

$\Rightarrow \text{抽到紅球的資訊量}$

$$I(p(\text{紅})) = -\log_2(0.5) = -(\textcolor{red}{-1}) = 1$$

$$(\textcolor{red}{2}^{\textcolor{red}{-1}} = 0.5)$$

編碼

資訊量 $I(p) = -\log_2(p)$

資訊量 = 需要幾個 bit 來傳遞結果

- 猜硬幣

- $I(\text{一次正面}) = -\log_2(0.5) = 1$

- $I(\text{兩次正面}) = -\log_2(0.5 * 0.5) = -\log_2(1/4) = -\log_2(2^{-2}) = -(-2) = 2$

- $I(\text{正、反、正}) = -\log_2(0.5 * 0.5 * 0.5) = -\log_2(1/8) = -\log_2(2^{-3}) = -(-3) = 3$

結果	正正正	正正反	正反正	正反反	反正正	反正反	反反正	反反反
編碼	000	001	010	011	100	101	110	111

- 要傳遞此系統的結果，需要用三位數 $\Rightarrow$ 3 個 bit

Entropy : 系統不確定性

資訊量 $I(p) = -\log_2(p)$

			
單場勝率	小傑 0.5	半藏 0.5	奇犽 0.75 爆庫兒 0.25
單場獲勝資訊量	1	1	0.415 2
系統不確定性	高		低
整體平均資訊量	$0.5 * 1 + 0.5 * 1 = 1$		$0.75 * 0.415 + 0.25 * 2 = 0.81$

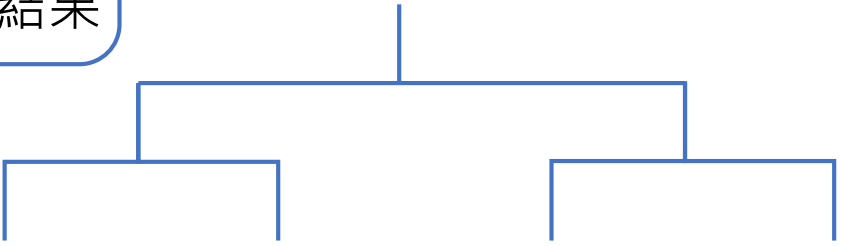
- Entropy (熵) 「混亂程度」

系統不確定性 = 意外程度的期望值

$$\begin{aligned} H(X) &= \sum p(x) \times I(x) \\ &= -\sum p(x) \log_2 p(x) \end{aligned}$$

Entropy : 所需 bit 數

資訊量 $I(p) = -\log_2(p)$
= 需要幾個 bit 來傳遞結果

				
冠軍機率	小傑	半藏	奇犽	爆庫兒
	0.25	0.25	0.25	0.25
得冠軍資訊量	2	2	2	2
Entropy	$0.25 * 2 + 0.25 * 2 + 0.25 * 2 + 0.25 * 2 = 2$			
編碼	00	01	10	11

- Entropy (熵) 「混亂程度」
系統不確定性 = 意外程度的期望值

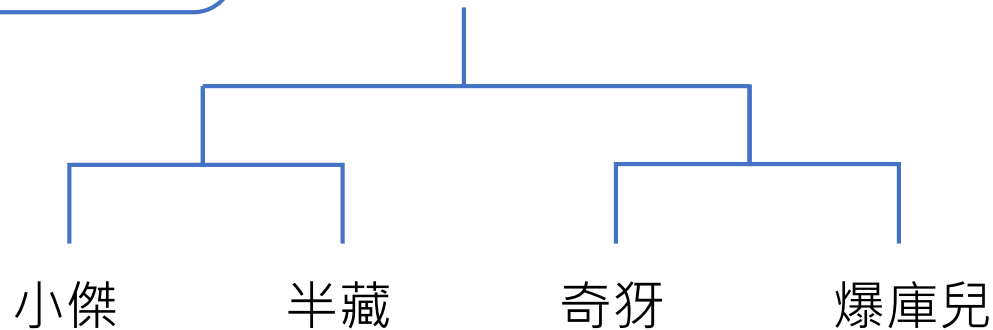
$$H(X) = \sum p(x) \times I(x)$$
$$= -\sum p(x) \log_2 p(x)$$

⇒ 傳遞「誰得冠軍」平均需要 2 bit

⇒ 2 個 bit

Entropy : 編碼效率

資訊量 $I(p) = -\log_2(p)$
= 需要幾個 bit 來傳遞結果



冠軍機率	1/2	1/4	1/8	1/8
得冠軍資訊量	1	2	3	3
Entropy	$1/2 * 1 + 1/4 * 2 + 1/8 * 3 + 1/8 * 3 = 1.75$			
編碼	0	10	110	111

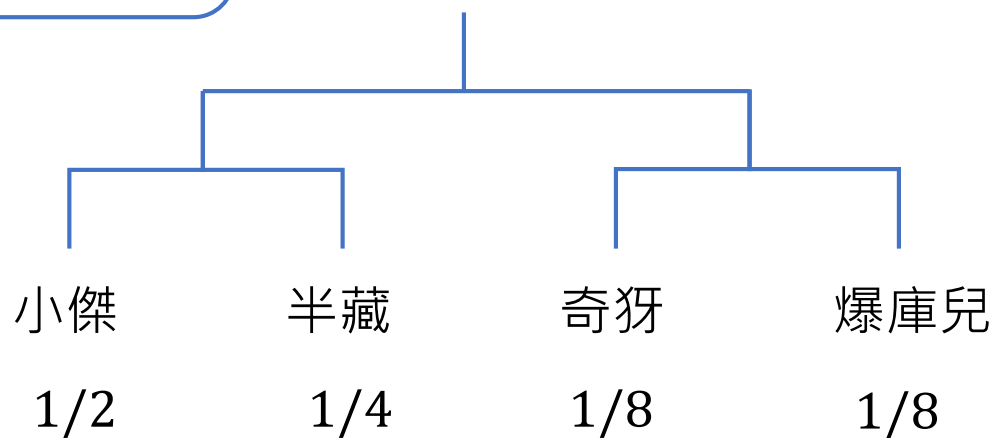
- Entropy (熵) 「混亂程度」
系統不確定性 = 意外程度的期望值

$$H(X) = \sum p(x) \times I(x)$$
$$= -\sum p(x) \log_2 p(x)$$

最有效率的編碼

Cross Entropy : 以錯誤的分布進行編碼

資訊量 $I(p) = -\log_2(p)$
= 需要幾個 bit 來傳遞結果



Cross Entropy (交叉熵)

- 對於分布的預測錯誤，導致浪費 bit
- 用 Q 編碼，去傳遞 P 的結果

$$H(P, Q) = -\sum p(x) \log_2 q(x)$$

Entropy (熵)

$$H(P) = -\sum p(x) \log_2 p(x)$$

最有效的碼率	$1/2 * 1 + 1/4 * 2 + 1/8 * 3 + 1/8 * 3 = 1.75$
選用編碼	00 01 10 11 (不知實際分布，假設勝率均等)
平均所需 bit 數	$1/2 * 2 + 1/4 * 2 + 1/8 * 2 + 1/8 * 2 = 2 = \text{Cross Entropy}$

Cross Entropy：以錯誤的分布進行編碼

資訊量 $I(p) = -\log_2(p)$
= 需要幾個 bit 來傳遞結果

$$H(P) \leq H(P, Q)$$

Cross Entropy (交叉熵)

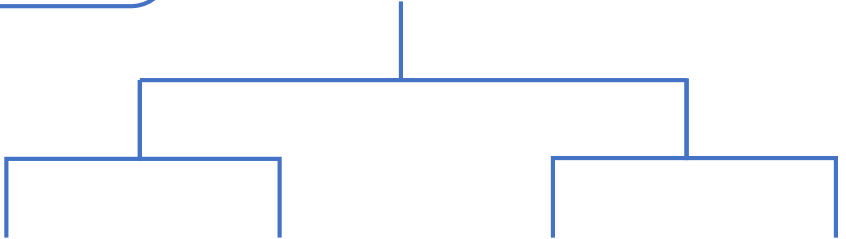
- 對於分布的預測錯誤，導致浪費 bit

用 Q 編碼，去傳遞 P 的結果

$$H(P, Q) = -\sum p(x) \log_2 q(x)$$

Entropy (熵)

$$H(P) = -\sum p(x) \log_2 p(x)$$

				
冠軍機率	小傑 1/4	半藏 1/4	奇犽 1/4	爆庫兒 1/4
最有效的碼率	$1/4 * 2 + 1/4 * 2 + 1/4 * 2 + 1/4 * 2 = 2$			
選用編碼	0	10	110	111
平均所需 bit 數	$1/4 * 1 + 1/4 * 2 + 1/4 * 3 + 1/4 * 3 = 2.25$			

(分布預測錯誤，採用較無效的編碼)

= Cross Entropy

KL Divergence : 多付出的代價

- $D_{KL}(P||Q) = H(P, Q) - H(P)$ (用錯誤分布編碼所需代價 - 理想編碼所需代價 = 多付的代價)
(分布 P 和 Q 的差異)

● 情境一	小傑	半藏	奇犽	爆庫兒		
冠軍機率	1/2	1/4	1/8	1/8	理想碼率 : 1.75	KL 散度 多付出的代價 0.25
選用編碼	00	01	10	11	實際碼率 : 2	
● 情境二	小傑	半藏	奇犽	爆庫兒		
冠軍機率	1/4	1/4	1/4	1/4	理想碼率 : 2	一般而言不會相等 0.25
選用編碼	0	10	110	111	實際碼率 : 2.25	

總結 - 資訊理論

- 資訊量 $I(p) = -\log_2(p)$ = 需要幾個 bit 來傳遞結果
- Entropy (熵) : $H(P) = -\sum p(x) \log_2 p(x)$
 - 系統不確定性 = 意外程度期望值 = 傳遞該系統所需平均 bit 數
- Cross Entropy (交叉熵) : $H(P, Q) = -\sum p(x) \log_2 q(x)$
 - 對於分布的預測錯誤，導致花費較多的 bit 數
- KL Divergence (KL 散度) : $D_{KL}(P||Q) = H(P, Q) - H(P)$
 - 用錯誤分布編碼所需多付的代價 (分布 P 和 Q 的差異)

關鍵特性

- $H(P)$ = 理想碼率
- $H(P, Q) \geq H(P)$
- $D_{KL}(P||Q) \geq 0$
- $D_{KL}(P||Q) = 0 \iff P = Q$

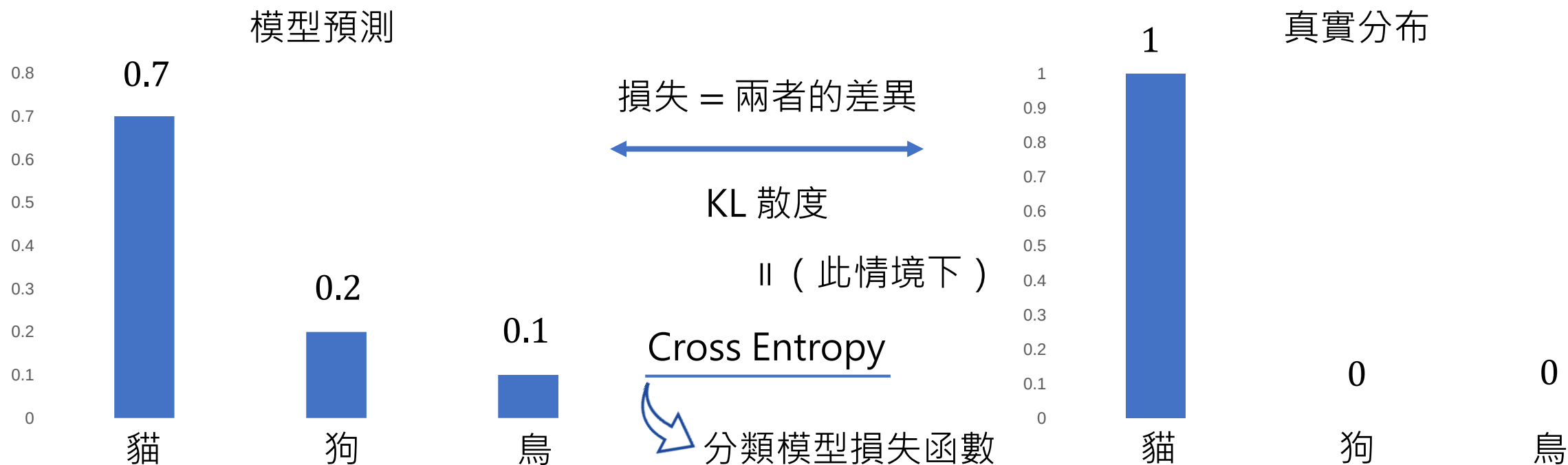
	小傑	半藏	奇犽	爆庫兒	
冠軍機率	1/2	1/4	1/8	1/8	理想碼率 : 1.75
選用編碼	00	01	10	11	實際碼率 : 2

0.25

分類模型損失函數

- 影像分類模型：吃一張圖片，輸出每個類別的機率（輸出向量，再過 Softmax）

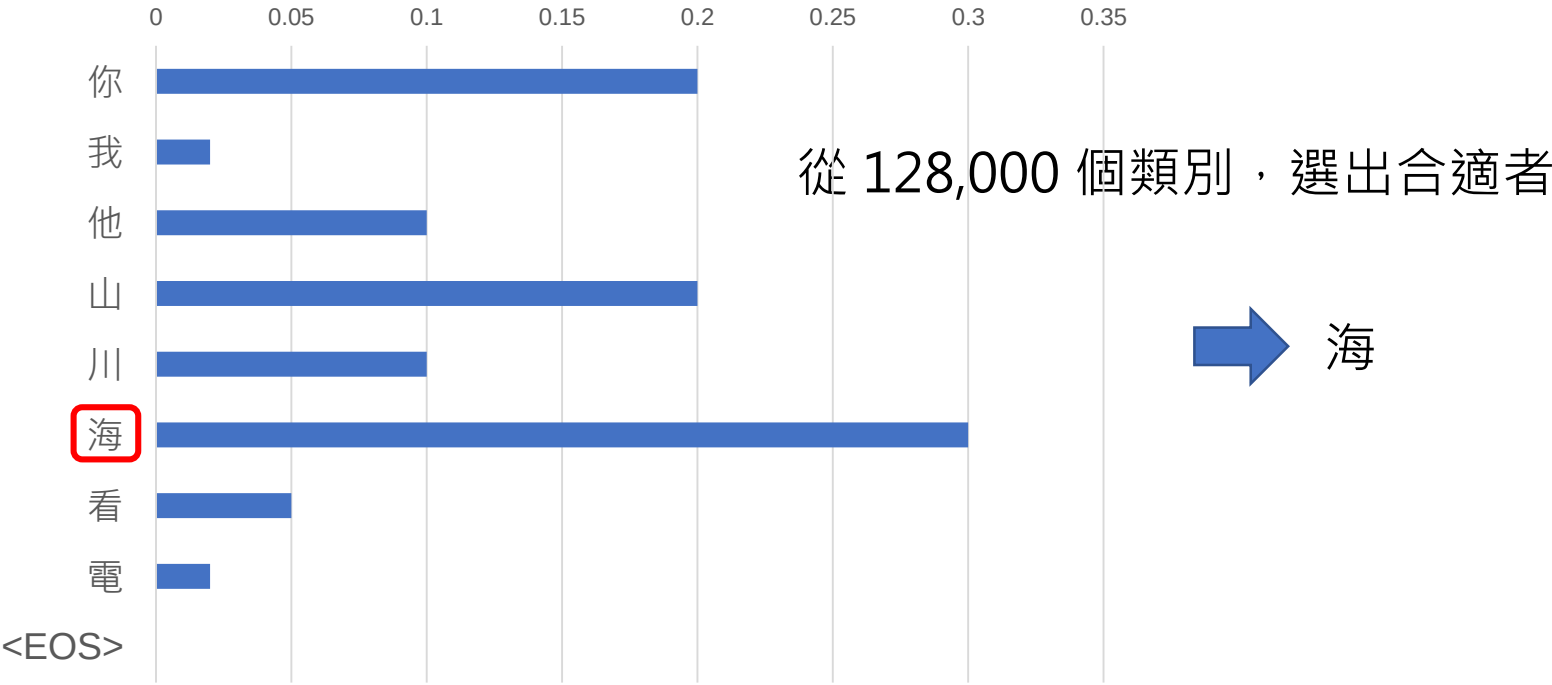
- $$D_{\text{KL}}(P\|Q) = H(P, Q) - \underbrace{H(P)}_0 = \sum_x P(x) \log\left(\frac{P(x)}{Q(x)}\right) = \boxed{1 \cdot \log\left(\frac{1}{0.7}\right)} + 0 \cdot \log\left(\frac{0}{0.2}\right) + 0 \cdot \log\left(\frac{0}{0.1}\right)$$
$$\approx 0.5146 = -1 \cdot \log(0.7)$$



語言模型運作 - 遞迴分類

語言模型 = 文字接龍 = 遞迴分類

損失：每生成一個 token，算一個 Cross Entropy



LLM

我 喜 歡

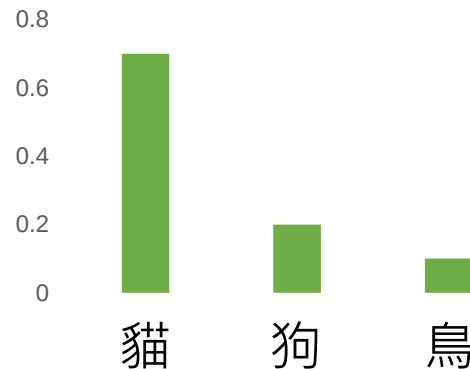
分類 vs. 生成

- 影像分類模型：吃一張圖片，輸出每個類別的機率



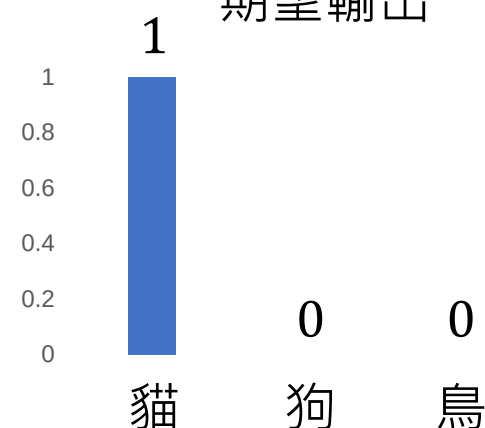
影像分類模型

模型輸出



CE

期望輸出

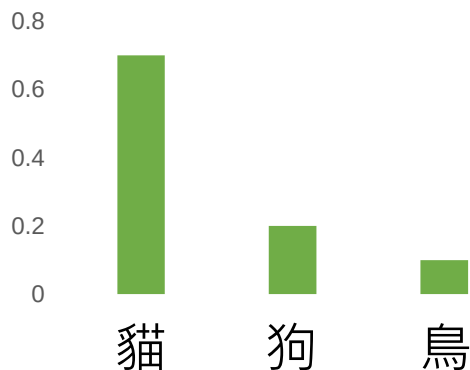


- 語言模型：吃一段文字，輸出「每個字成為下一個字的機率」

我喜歡

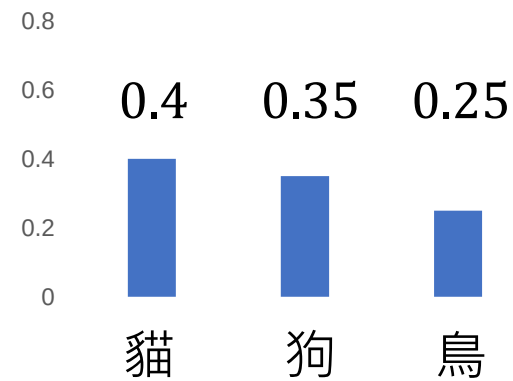
語言模型

模型輸出



CE

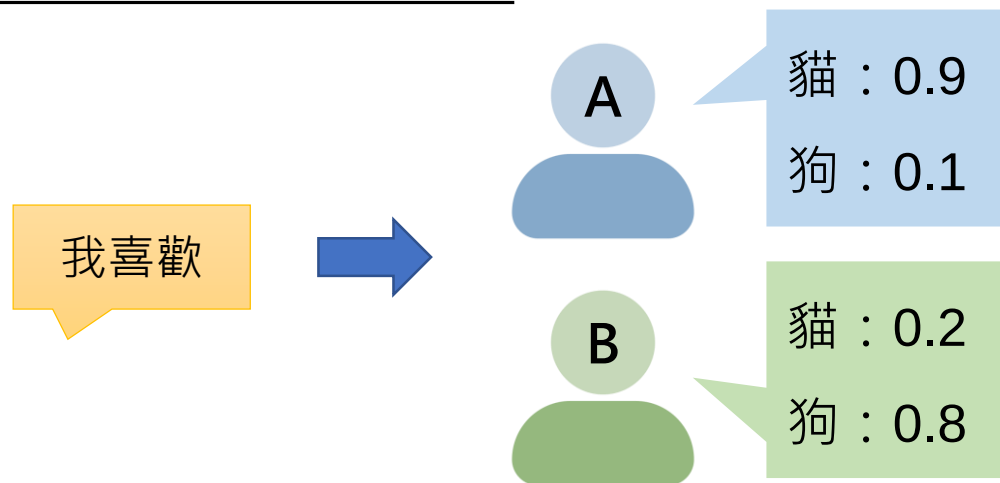
期望輸出



沒有唯一正確答案

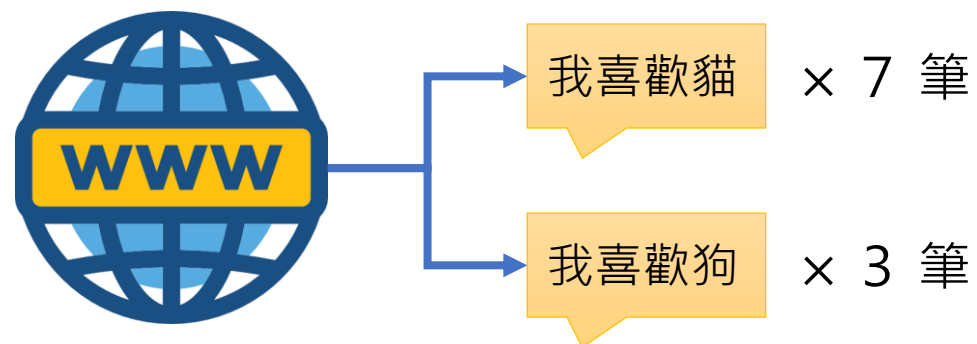
定義語言的理想分布

如何定義「理想分布」

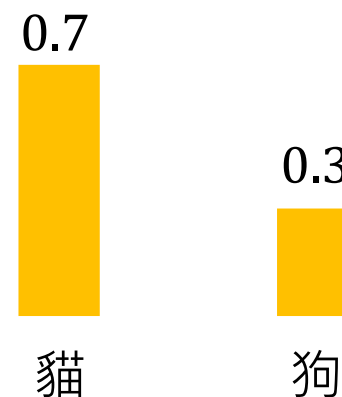


需有客觀、易取得的目標分布

- 把網路 / 語料庫當中出現的頻率，當成目標分布



目標分布



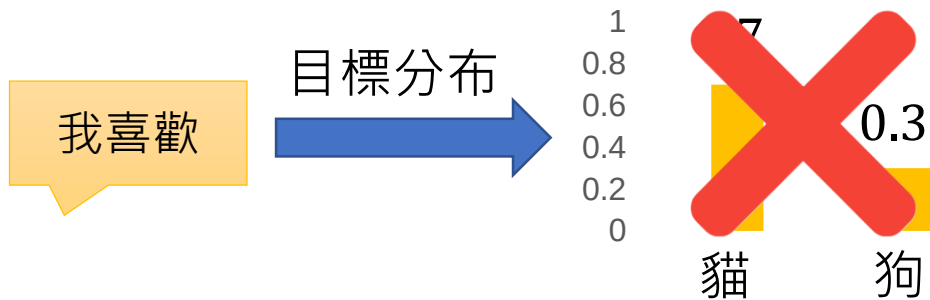
抽樣偏誤

資料中的行為分布

|| ?

世界的真實分布

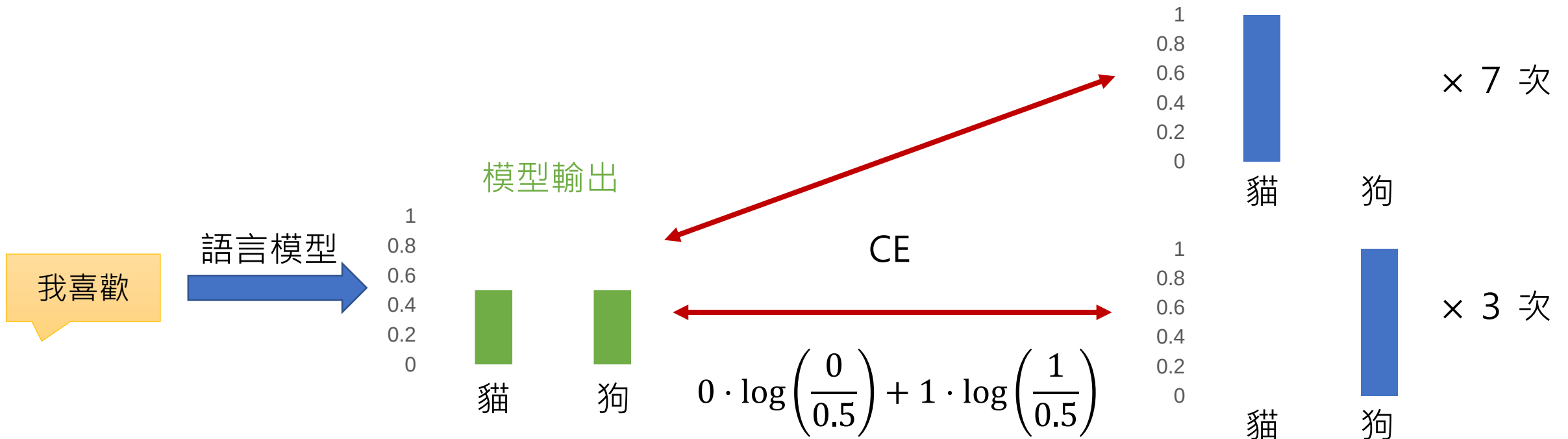
學習語言的理想分布



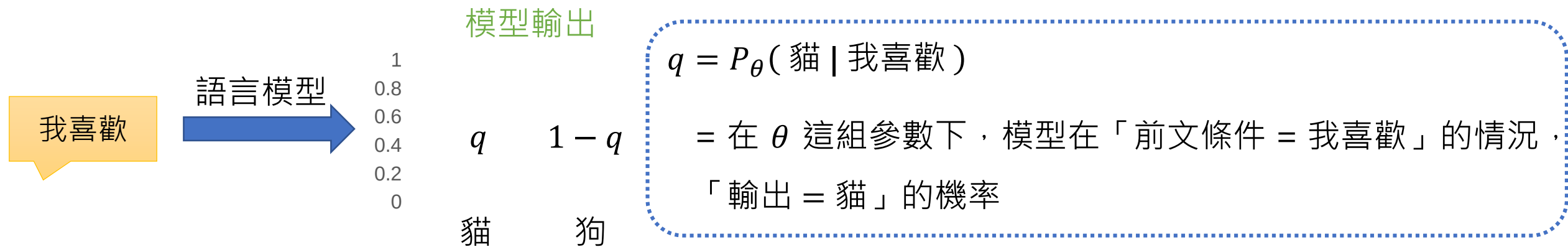
- 無法先統計出資料中所有上文 (Context) 的下一詞分布

↓ 透過抽樣學習分布

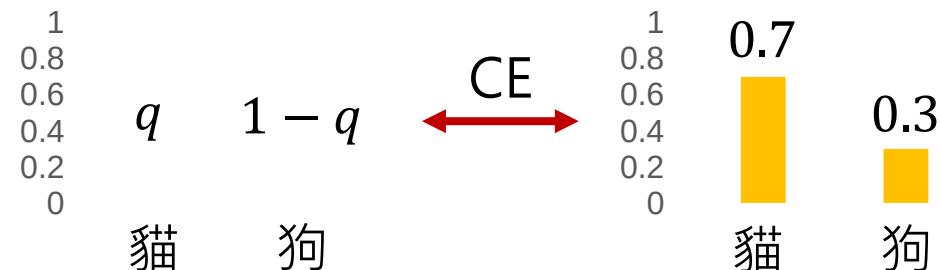
- 把資料從頭讀到尾，讀到什麼就學什麼（每筆都說 100%）



從 One-Hot 學習的合理性



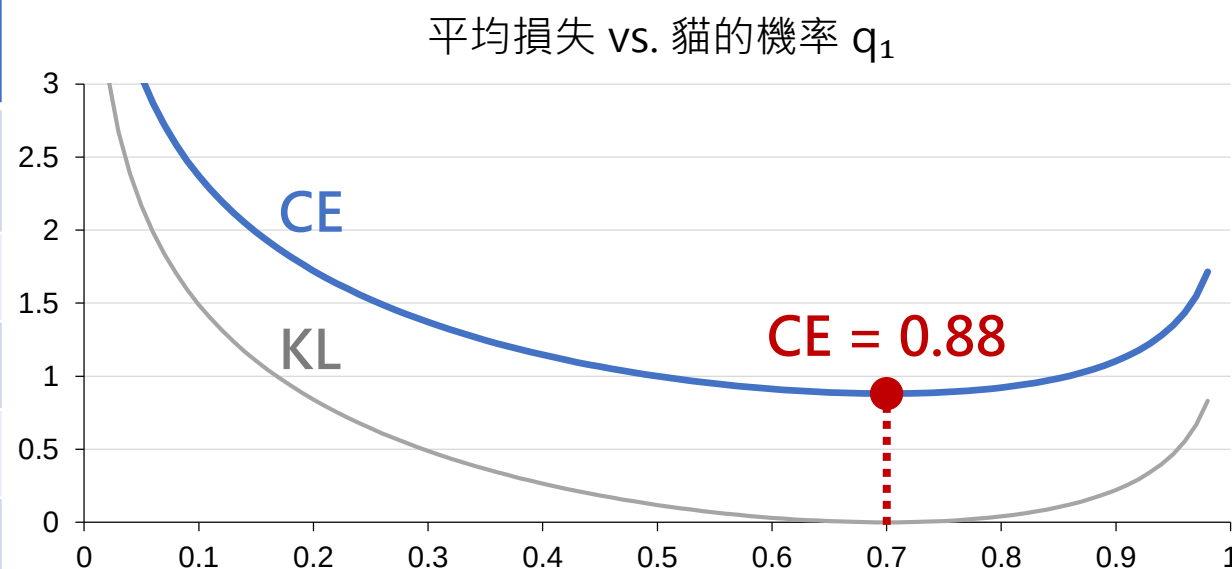
- 1 筆 One-Hot 貓的損失 = $-\log p$; 1 筆 One-Hot 狗的損失 = $-\log(1 - p)$
- 10 筆 One-Hot 的總損失 = $L(p) = -7 \log p - 3 \log(1 - p)$
- 10 筆 One-Hot 的平均損失 = $\frac{L(p)}{10} = -0.7 \log p - 0.3 \log(1 - p)$ = 以 (0.7, 0.3) 為目標的 CE



Loss 最小發生點

- 平均損失 = $0.7 \cdot (-\log_2 q_1) + 0.3 \cdot (-\log_2 q_2) =$ 以 $(0.7, 0.3)$ 為目標的 CE
- CE = 以預測的分布進行編碼所需 bit 數 $\Rightarrow$ CE 最小值發生於預測分布 $(q_1, q_2) =$ 實際分布 $(0.7, 0.3)$

模型輸出 (貓, 狗)	平均 CE 損失 (bit)	平均 KL 損失 (bit)
(1.0, 0.0)	∞ (狗的損失是 $-\log_2 0$)	∞
(0.9, 0.1)	1.10	0.22
(0.7, 0.3)	0.88	0
(0.5, 0.5)	1.00	0.12
(0.3, 0.7)	1.37	0.49



每一筆資料都說「100%」，模型卻能學到「70%」

每筆資料是對世界的一次抽樣：one-hot 是抽樣結果，不是目標；目標是抽樣背後的分佈

Loss 的下限：語言本身的 Entropy

- $CE = Entropy + KL \Rightarrow CE$ 最小值發生於 $KL = 0$ ，即「預測分布 $Q =$ 實際分布 P 」
- **CE 損失最小值 = Entropy = 0.88 bit**
- Entropy 代表資料當中的「語言的不確定性」，與模型無關

影像分類

P 是 one-hot, Entropy = 0
 $\Rightarrow$ CE 損失可以降到 0

語言模型

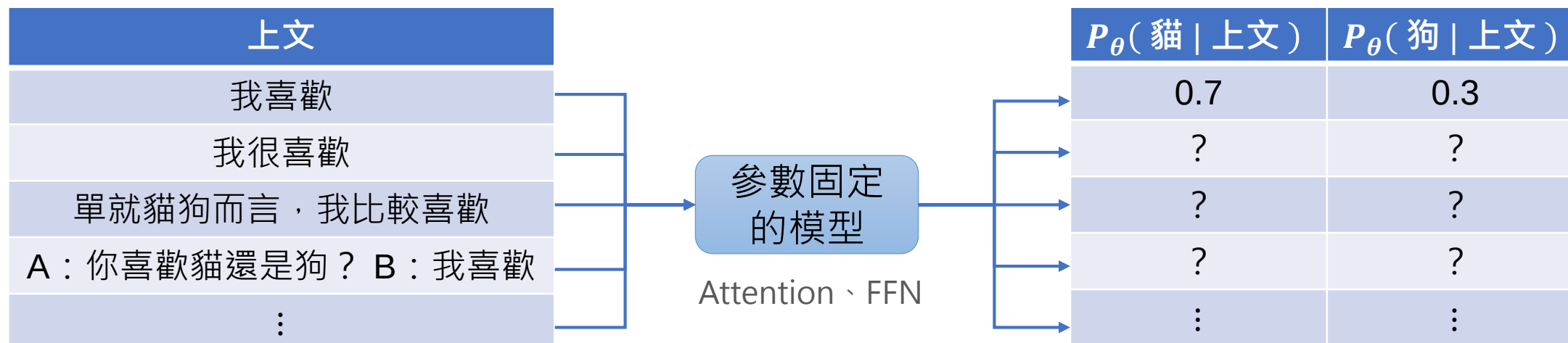
P 不是 one-hot, Entropy > 0
 $\Rightarrow$ CE 損失永遠降不到 0
因為語言本身就有不確定性

- $Perplexity = 2^{CE} = 2^{0.88} \approx 1.84 \Rightarrow$ 有 1.84 個「等效選項數」
(困惑度, PPL) (困惑程度相當於「在 1.84 個選項之間做選擇」)

模型泛化性

模型不是資料庫，而是一個共用的函數

- 如果模型只是背下「每個前文 → 下一個字的分布」，它就是一張巨大的查表



- 模型不可能儲存「所有可能的前文」的下一個字的分布
- 模型推論時收到的輸入，幾乎沒有一個曾經在訓練資料裡一模一樣出現過

模型學的不是表，是規律

- 學習一套通用的方式，對任何不同輸入都能提取語意、做接龍

$$f_{\theta}(x) = P(\text{next token} | x)$$

大模型仍會有「記憶資料」的行為，但「學習規律」是更有效利用參數、降低損失的方式

預測即壓縮

- Entropy：用最理想的編碼傳遞結果，所需的 bit 數
- 語言模型的損失單位就是 **bit / 字**
CE 損失 0.88 $\Rightarrow$ 拿這個模型當編碼器，每個字平均只需 0.88 bit
- 目標：用**有限模型參數**在**大量資料**上取得**低損失** $\Rightarrow$ 能有效壓縮各種文字 / 語句 / 文章
 $\Rightarrow$ 只有一條路能達成：**抓到背後的規律**（文法、常識、事實、推理等）
- 在沒見過的文字上損失低 = 能壓縮沒見過的文字 $\Rightarrow$ 能預測沒見過的文字

類比：傳遞數列

1, 1, 2, 3, 5, 8, 13, 21, 34, ... $\Rightarrow$ 需用大量 bits 儲存 / 傳輸 ; 只能記住歷史資料

$\Downarrow$ 提取規律 / 特徵

$a_i = a_{i-1} + a_{i-2}$ $\Rightarrow$ 壓縮成少量 bits 儲存 / 傳輸 ; 可以**預測**後續資料

壓縮即智能？

我把冰淇淋放在夏日的陽光下，
半小時後，冰淇淋 _____

- 冰淇淋物理性質
- 太陽會加熱物體
- 夏天通常很熱
- 半小時足夠長
- 「融化」這個概念
- 因果關係

- 預測 next token = 壓縮資料（用短的描述取代長的資料）

↑

↓
把資料壓縮到極致，有時
模型確實會被迫學到模式

- 從資料中學習出「可重複使用」的**規律 / 模式**，就能做好壓縮

文法 / 語意 / 常識 / 物理規律 /
關係 / 推理 / 抽象概念

↑

- 有足夠「**智能**」，就能從資料中學習出規律 / 模式

做好預測 next token $\Rightarrow$ 有足夠智能？

課程總結

線性代數

模型架構（能力）

- 向量表示、矩陣乘法
- embedding、attention、FFN
- 決定模型「能表達什麼函數」

微積分

梯度下降（搜尋）

- 導數、chain rule
- 反向傳播
- 決定「怎麼找到參數」

統計

學習目標（方向）

- 機率分布、期望值
- Cross Entropy、KL 散度
- 決定「什麼叫做學得好」

深度學習 = 用線性代數搭出來的函數，朝著統計定義的目標，用微積分一步一步走過去

遇到新模型、新方法，問三個問題：它的架構是什麼？它的目標是什麼？它怎麼優化？